

Full Length Article

Fractional-order gradient descent learning for Elman neural networks^{*}He Li^{a,*}, Shanze Wang^a, Yangquan Chen^b, Yanmei Liu^a^a Shenyang Aerospace University, No.37 Daoyi South Avenue, Daoyi District, Shenyang, 110136, Liaoning Province, China^b MESA Lab, Dept. of Mechanical and Aerospace Engineering, University of California Merced, CA, 95343, USA

ARTICLE INFO

Keywords:

Fractional-order gradient descent
 Elman neural network
 Grünwald-Letnikov definition
 System identification
 Time series prediction
 Recurrent neural network

ABSTRACT

To address the limitations of conventional integer-order gradient descent in training Elman neural networks, such as susceptibility to local minima and slow convergence-this paper proposes a fractional-order gradient descent learning algorithm for Elman networks based on the Grünwald-Letnikov definition (FO-Elman). First, the fractional-order gradient expressions for each layer of the Elman network are systematically derived, and a complete backpropagation framework is established; the convergence of the proposed algorithm is also proved theoretically. Then, by exploiting the memory property of fractional calculus, the proposed method incorporates a weighted aggregation of historical gradient information into the parameter update rule, thereby mitigating the shortcomings of standard optimization schemes. Finally, experimental results on system identification and time-series prediction tasks demonstrate that FO-Elman achieves improved optimization performance for Elman networks, providing a new theoretical and algorithmic tool for recurrent neural network training.

1. Introduction

Neural networks, as a core technology in the fields of machine learning and artificial intelligence, have demonstrated outstanding performance in numerous applications such as pattern recognition, system identification, and time series prediction (Cao et al., 2006; Gawlikowski et al., 2023; Soydaner, 2022). Among them, recurrent neural networks (RNNs) exhibit unique advantages in processing sequential data and temporal-related problems due to their inherent memory mechanisms and dynamic characteristics (Amalou et al., 2022; Saon et al., 2021; Wang et al., 2022). The Elman neural network, as an early and classical RNN architecture (Liou et al., 2008), features a simple structure and high computational efficiency. By introducing a recursive layer in the hidden layer to achieve state feedback, it provides an effective solution for processing temporal information.

Currently, RNNs have been widely applied across various domains, including industrial process control (Bonassi et al., 2022; Dhaked, 2024; Lanzetti et al., 2019; Sharma et al., 2021), financial market prediction (Hansun & Young, 2021), speech signal processing (Gelly & Gauvain, 2018; Zhao et al., 2017), biomedical signal analysis (Lyu et al., 2017; Naderi & Mahdavi-Nasab, 2010), traffic flow prediction (Tian & Pan, 2015), and power system load forecasting (Bao Huy et al., 2023; Cui et al., 2020; Kong et al., 2019), among many other practical application scenarios. Beyond forecasting, Dhaked et al. demonstrated that intelligent decision schemes can also support grid protection by identifying

and classifying transmission-line faults using a fuzzy logic controller with high-speed digital relays (Dhaked et al., 2023b). In the field of system identification, RNNs have been successfully employed for black-box modeling of nonlinear dynamic systems (Fei & Lu, 2018; Kosmatopoulos et al., 1992; Kumar, 2024; Kuschewski et al., 1993; Wang & Chen, 2006), such as chemical reactor modeling (Zarzycki & Ławryńczuk, 2021), robotic dynamics identification (Folgheraiter et al., 2023), and parameter estimation for complex hydraulic systems (Liu et al., 2025). Related data-driven identification has also been explored for electrochemical energy systems; Dhaked et al. compared multiple machine-learning approaches for real-time PEM fuel cell voltage prediction and early-stage deterioration forecasting (Dhaked et al., 2024). In time series prediction (Atabay et al., 2022; Dhaked et al., 2023a; King et al., 2024), these networks perform exceptionally well in handling sequence data with long-term dependencies, achieving remarkable results in applications like weather forecasting (Zhao et al., 2018), stock price fluctuation prediction (Zhao et al., 2021), and network traffic prediction (Ramakrishnan & Soni, 2018).

Numerous excellent variants of RNNs have been developed, including long short-term memory networks (LSTM) (Fu et al., 2024; Greff et al., 2017; Zhang et al., 2024a), gated recurrent units (GRU) (Dey & Salem, 2017), and various attention mechanism networks (Feng et al., 2024). Researchers have combined them with other neural networks and algorithms. For instance, Lakshmi Narayanan et al. designed a machine-learning-based pitch controller for large-scale wind turbines using an

^{*} This work was supported by the National Natural Science Foundation of China under Grant 62003223.

^{*} Corresponding author.

E-mail addresses: lihe_good@126.com (H. Li), wangshanze0603@163.com (S. Wang), ychen53@ucmerced.edu (Y. Chen), 1154318449@qq.com (Y. Liu).

improved recurrent radial basis function neural network, with parameters tuned by a modified particle swarm optimization algorithm to achieve more stable rated power generation (Lakshmi Narayanan et al., 2024). Zha et al. (2022) integrated the powerful feature extraction capability of convolutional neural networks (CNN) with the superior temporal modeling advantages of LSTM to construct a CNN-LSTM hybrid model, which achieved better performance than standalone LSTM in predicting monthly gas field production. Sun et al. (2024) enhanced this hybrid model by adding an attention mechanism, proposing the CNN-LSTM-AM model to predict offshore wind turbine power using signals from multiple sensors. Through controlled variable comparisons, they completed sensor sensitivity analysis to identify the optimal sensor group for power prediction. Liu et al. (2021) proposed a lithology sequence classification algorithm based on bidirectional gated recurrent units and a conditional random field layer (BiGRU-CRF). They concatenated the probability output vector of a support vector machine (SVM) with the original data as input to BiGRU-CRF, demonstrating that the SVM-BiGRU-CRF algorithm generally achieves the best results regardless of whether the training and test sequences are similar or dissimilar. Zhang et al. (2024b) addressed the time lag issue in LSTM prediction by proposing a hybrid model coupling empirical/variational mode decomposition (E/VMD) with LSTM. VMD separates high-frequency noise from slowly varying low-frequency components, enhancing the stationarity of the original sequence. The model was validated using measured significant wave height and wind speed data, effectively mitigating the lag problem in LSTM.

Despite the development of such rich variants and advanced hybrid models of RNNs, the Elman neural network maintains its importance in many practical applications due to its simple structure, fewer parameters, and fast training speed (Huang & Qin, 2024; Liu et al., 2015). Particularly in systems with limited computational resources, real-time control applications, and modeling tasks with small to medium-sized datasets, the efficiency and interpretability of the Elman network make it the preferred solution. Furthermore, as a fundamental architecture of RNNs, the Elman network serves as an ideal testbed for researching and validating new optimization algorithms. Its simple structure facilitates in-depth analysis of algorithm convergence properties and optimization mechanisms. However, as application scenarios become increasingly complex and demands for prediction accuracy continue to rise, the traditional gradient descent learning algorithm used in Elman networks gradually reveals inherent limitations when addressing complex nonlinear optimization problems. These include susceptibility to local optima, slow convergence, sensitivity to initial parameter settings, and potential oscillation phenomena on complex loss function surfaces (Liu et al., 2022; Zhang et al., 2021b). These drawbacks severely restrict the application effectiveness of Elman networks in high-precision modeling and prediction tasks.

With the continuous development of deep learning theory and significant improvements in computational power, researchers have begun integrating fractional calculus with neural networks to explore more flexible and efficient learning algorithms. Fractional calculus, as an extension of classical calculus, allows for arbitrary real-number orders rather than traditional positive integers. This characteristic endows fractional-order algorithms with greater flexibility and memory properties (Chen et al., 2023, 2021; Kang et al., 2025; Lopes Ferrari et al., 2024; Sivalingam & Govindaraj, 2025; Wang et al., 2017; Yuan et al., 2026). Chen et al. (2024) proposed a fractional-order gradient descent optimizer based on the Caputo operator and Adam algorithm, designing a fractional-order scheduler (FOS) based on the connection cloud model (CCM). This approach achieved better classification performance than traditional gradient descent algorithms in the identification and classification tasks of logging core image datasets. Other researchers have combined fractional calculus with neural networks (Zuñiga Aguilar et al., 2020), proposing a fractional-order neural network specifically for system identification. The effectiveness of this new black-box modeling method was validated through the identification of three different

systems. Pang et al. (2019) extended physics-informed neural networks (PINN) to fractional-order PINN (FPINN) to solve space-time fractional advection-diffusion equations, systematically investigating their convergence and demonstrating their accuracy and effectiveness in solving multidimensional forward and inverse problems with forcing terms. Yao and Wang (2020) proposed a fractional-order echo state network (FESN) for time series prediction. For FESN, the reservoir state differential equation was replaced with a fractional-order differential equation, and its effectiveness was validated through two numerical examples. Fractional calculus theory can effectively extend traditional activation functions to fractional-order activation functions (Bingi et al., 2025), with related studies showing that this improvement significantly enhances the accuracy and robustness of wind power prediction models. Vellappandi and Lee (2025) provided an innovative approach for modeling complex dynamical systems by integrating Caputo fractional derivatives into deep neural networks, effectively capturing memory effects and long-term dependencies in such systems.

Existing research indicates that fractional-order gradient descent methods have the potential to surpass traditional integer-order methods in optimizing neural networks (Liu et al., 2018; Zhang et al., 2021a). However, for the Elman network—a fundamental RNN with unique application advantages—the training process still primarily relies on integer-order gradient descent algorithms with significant limitations. To overcome these constraints, this paper proposes a novel fractional-order gradient descent learning algorithm for Elman neural networks. This algorithm incorporates the fractional calculus theory defined by Grünwald-Letnikov into the error backpropagation framework of Elman networks. By designing gradient computation and weight update rules with fractional-order characteristics, it aims to effectively mitigate local optima traps during training, accelerate convergence, and ultimately enhance the network's performance in temporal modeling and prediction tasks. The main contributions of this study are as follows:

1. A Grünwald-Letnikov-based fractional-order gradient descent algorithm is developed for Elman neural network training, introducing a memory-aware update rule based on historical gradients.
2. A complete FO-Elman backpropagation framework is derived with layer-wise fractional gradients, and its convergence is established under Lipschitz continuity and strong convexity; the role of α is further analyzed via weight-decay characteristics.
3. Extensive experiments on system identification and time series prediction benchmark FO-Elman against GD/SGDM/RMSProp/Adam and recurrent baselines (LSTM/GRU), together with a parameter sensitivity analysis, showing consistent MSE improvements.

The remainder of this paper is organized as follows. Section 2 introduces the Grünwald-Letnikov-based fractional-order gradient descent algorithm (FOGD) and its basic properties. Section 3 presents the FO-Elman learning framework and convergence analysis. Section 4 reports experiments on system identification and time series prediction, including baseline comparisons and a parameter sensitivity study. Section 5 concludes the paper.

2. Preliminaries

This section presents the derivation and simulation verification of the Fractional-Order Gradient Descent (FOGD) algorithm based on the Grünwald-Letnikov definition.

2.1. Fractional-order derivatives via Grünwald-Letnikov definition

Fractional calculus, as a generalization of classical calculus, demonstrates unique advantages in optimization algorithm design. Unlike traditional integer-order derivatives, fractional derivatives possess memory properties that retain historical information, enabling fractional-order-based optimization algorithms to exhibit superior

convergence performance and global search capabilities when handling complex nonlinear problems.

The Grünwald-Letnikov (G-L) definition of fractional derivatives provides the theoretical foundation for optimization algorithm design. For a function $f(t)$ defined on the interval $[a, b]$, its α -order fractional derivative at point t is defined as:

$$D_a^\alpha f(t) = \lim_{h \rightarrow 0} h^{-\alpha} \sum_{j=0}^{\lfloor (t-a)/h \rfloor} (-1)^j \binom{\alpha}{j} f(t - jh), \quad (1)$$

where $h > 0$ is the step size, a is the starting point, $t \geq a$ is the current point, $\alpha > 0$ is the order of the fractional derivative, $\lfloor \cdot \rfloor$ denotes the floor function, $\binom{\alpha}{j} = \frac{\Gamma(\alpha+1)}{\Gamma(j+1)\Gamma(\alpha-j+1)}$ represents the generalized binomial coefficient, and $\Gamma(\cdot)$ is the Gamma function. This definition reveals that the value of the fractional derivative at each point depends on a weighted combination of all historical information preceding that point, with weights determined by the generalized binomial coefficients, reflecting the inherent non-locality and memory characteristics of fractional derivatives.

Several definitions of fractional derivatives exist, such as the Grünwald-Letnikov, Riemann-Liouville, and Caputo forms. We adopt the Grünwald-Letnikov definition because it leads to a discrete weighted-sum representation with explicit coefficients $\omega_j^{(\alpha)}$, which directly fits the iterative learning setting and enables an efficient truncated-memory implementation with length M . Under standard smoothness conditions, it is also consistent with the Riemann-Liouville definition, while the Caputo form is mainly advantageous for fractional differential equations with initial conditions and is not essential for the gradient-history aggregation used in this paper.

2.2. Fractional-order gradient descent algorithm

Gradient descent is one of the most fundamental and widely applied optimization algorithms in machine learning and deep learning, whose core idea involves using first-order derivative information of the loss function to iteratively update model parameters.

For an optimization problem, let $L(\theta)$ be the loss function and $\theta \in \mathbb{R}^d$ be the parameter vector to be optimized. The parameter update rule of conventional gradient descent is:

$$\theta_{k+1} = \theta_k - \eta \nabla L(\theta_k), \quad (2)$$

where $\eta > 0$ is the learning rate and $\nabla L(\theta_k)$ is the gradient of the loss function at the k th step parameter θ_k . In practice, the true gradient is typically estimated using mini-batch samples:

$$\nabla L(\theta_k) \approx \frac{1}{|B_k|} \sum_{i \in B_k} \nabla \ell_i(\theta_k), \quad (3)$$

where B_k denotes the mini-batch sample set at the k th step, and $\ell_i(\theta_k)$ represents the loss function for the i th sample.

The traditional integer-order gradient $\nabla L(\theta_k)$ is essentially a local linear approximation that utilizes only current point information to determine the optimization direction. Mathematically, the first-order derivative reflects the instantaneous rate of change of the gradient function at a point:

$$\nabla L(\theta) = \lim_{n \rightarrow 0} \frac{L(\theta + n) - L(\theta)}{n}. \quad (4)$$

This local property causes traditional gradient descent to consider only current gradient information during each update, completely disregarding historical optimization paths. Consequently, the algorithm is prone to becoming trapped in local optima, may exhibit oscillatory behavior in regions with complex loss function landscapes, and is sensitive to parameter initialization. To overcome these limitations, fractional calculus provides new theoretical tools for optimization algorithm design, with its natural memory properties enabling effective utilization of historical gradient information to improve optimization performance.

For discrete optimization problems, we consider the fractional-order gradient of the loss function $L(\theta_k)$ with respect to parameter θ at the k th step. The fractional-order gradient based on the Grünwald-Letnikov definition can be expressed as:

$$\nabla^\alpha L(\theta_k) = \sum_{j=0}^k \omega_j^{(\alpha)} \nabla L(\theta_{k-j}), \quad (5)$$

where the weight coefficients $\omega_j^{(\alpha)}$ are defined as:

$$\omega_j^{(\alpha)} = (-1)^j \frac{\Gamma(\alpha+1)}{\Gamma(j+1)\Gamma(\alpha-j+1)}. \quad (6)$$

Remark 1. The coefficients $\{\omega_j^{(\alpha)}\}_{j \geq 0}$ define a long-memory weighting on past gradients. Consequently, the fractional-order gradient can be interpreted as a weighted accumulation of historical gradients, where the influence of older gradients decays with j in a non-exponential manner determined by α .

These weight coefficients possess the following important properties: when $j = 0$, $\omega_0^{(\alpha)} = 1$; when $j \rightarrow \infty$, $\omega_j^{(\alpha)} \rightarrow 0$, demonstrating memory decay characteristics. Furthermore, the weight coefficients satisfy the recurrence relation:

$$\omega_j^{(\alpha)} = \frac{\alpha - j + 1}{j} \omega_{j-1}^{(\alpha)}, \quad j \geq 1. \quad (7)$$

Considering computational complexity and storage limitations, practical applications require truncation of the memory length. Let M be the memory length; the truncated fractional-order gradient becomes:

$$\nabla^\alpha L(\theta_k) = \sum_{j=0}^{\min(k, M-1)} \omega_j^{(\alpha)} \nabla L(\theta_{k-j}). \quad (8)$$

Based on the truncated fractional-order gradient, the parameter update rule for FOGD is:

$$\theta_{k+1} = \theta_k - \eta \sum_{j=0}^{\min(k, M-1)} \omega_j^{(\alpha)} \nabla L(\theta_{k-j}), \quad (9)$$

where η is the learning rate and $\alpha \in (0, 1)$ is the order of the fractional derivative.

The update direction in FOGD is constructed by combining the current gradient with a weighted contribution from historical gradients. This provides a continuous connection to integer-order methods through limiting cases. When $\alpha \rightarrow 0$, the update reduces to the conventional gradient descent that uses only the current gradient. When $\alpha \rightarrow 1$, the update approaches a gradient-difference type correction dominated by the most recent two gradients. Therefore, α adjusts the relative emphasis between recent and historical gradients, and the memory length M determines how many past gradients are effectively involved in practice.

Remark 2. The fractional order α determines the decay pattern of $\omega_j^{(\alpha)}$ and thus the strength of long-memory effect, while M sets the practical truncation horizon of the gradient history.

To analyze the convergence of the FOGD algorithm, we introduce the following standard assumptions: the gradient of the loss function $L(\theta)$ is Lipschitz continuous, i.e., there exists a constant $L > 0$ such that

$$\|\nabla L(\theta_1) - \nabla L(\theta_2)\| \leq L \|\theta_1 - \theta_2\|, \quad \forall \theta_1, \theta_2; \quad (10)$$

the loss function $L(\theta)$ is μ -strongly convex, i.e., there exists a constant $\mu > 0$ such that

$$(\theta_1 - \theta_2)^T (\nabla L(\theta_1) - \nabla L(\theta_2)) \geq \mu \|\theta_1 - \theta_2\|^2, \quad \forall \theta_1, \theta_2; \quad (11)$$

the gradient noise is bounded, i.e., there exists a constant $\sigma^2 \geq 0$ such that

$$\mathbb{E}[\|\nabla L(\theta_k) - \nabla L(\theta^*)\|^2] \leq \sigma^2, \quad \forall k, \quad (12)$$

where θ^* is the global optimal solution of the loss function satisfying $\nabla L(\theta^*) = 0$; and the boundedness of weight coefficients, i.e., for

$\alpha \in (0, 1)$, define

$$S_\alpha = \sum_{j=0}^{\infty} |\omega_j^{(\alpha)}|^2 < \infty, \quad T_\alpha = \sum_{j=1}^{\infty} |\omega_j^{(\alpha)}|^2 < \infty. \quad (13)$$

Based on these assumptions to prove the convergence of the FOGD algorithm, we construct the energy function $V_k = \mathbb{E}[\|\theta_k - \theta^*\|^2]$ to analyze the algorithm's convergence behavior, where θ^* is the global optimal solution of the loss function.

According to the FOGD update rule, denoting the fractional-order gradient as $G_k^{(\alpha)} = \sum_{j=0}^{\min(k, M-1)} \omega_j^{(\alpha)} \nabla L(\theta_{k-j})$, we have:

$$\begin{aligned} \mathbb{E}[\|\theta_{k+1} - \theta^*\|^2] &= \mathbb{E}[\|\theta_k - \eta G_k^{(\alpha)} - \theta^*\|^2] \\ &= \mathbb{E}[\|\theta_k - \theta^*\|^2] - 2\eta \mathbb{E}[(\theta_k - \theta^*)^T G_k^{(\alpha)}] \\ &\quad + \eta^2 \mathbb{E}[\|G_k^{(\alpha)}\|^2]. \end{aligned} \quad (14)$$

For the fractional-order gradient term, we decompose it as:

$$\begin{aligned} (\theta_k - \theta^*)^T G_k^{(\alpha)} &= (\theta_k - \theta^*)^T \nabla L(\theta_k) \\ &\quad + \sum_{j=1}^{\min(k, M-1)} \omega_j^{(\alpha)} (\theta_k - \theta^*)^T \nabla L(\theta_{k-j}). \end{aligned} \quad (15)$$

For the main term, using the strong convexity assumption (11), we obtain:

$$(\theta_k - \theta^*)^T \nabla L(\theta_k) \geq \mu \|\theta_k - \theta^*\|^2 + (\theta_k - \theta^*)^T \nabla L(\theta^*). \quad (16)$$

Since θ^* is the optimal solution, $\nabla L(\theta^*) = 0$, thus:

$$(\theta_k - \theta^*)^T \nabla L(\theta_k) \geq \mu \|\theta_k - \theta^*\|^2. \quad (17)$$

For the historical terms, we have:

$$\begin{aligned} &\left| \sum_{j=1}^{\min(k, M-1)} \omega_j^{(\alpha)} (\theta_k - \theta^*)^T \nabla L(\theta_{k-j}) \right| \\ &\leq \|\theta_k - \theta^*\| \sum_{j=1}^{\min(k, M-1)} |\omega_j^{(\alpha)}| \|\nabla L(\theta_{k-j})\| \end{aligned} \quad (18)$$

For any $\varepsilon > 0$, by Young's inequality:

$$\begin{aligned} &\|\theta_k - \theta^*\| \sum_{j=1}^{\min(k, M-1)} |\omega_j^{(\alpha)}| \|\nabla L(\theta_{k-j})\| \\ &\leq \frac{\varepsilon}{2} \|\theta_k - \theta^*\|^2 + \frac{1}{2\varepsilon} \left(\sum_{j=1}^{\min(k, M-1)} |\omega_j^{(\alpha)}| \|\nabla L(\theta_{k-j})\| \right)^2. \end{aligned} \quad (19)$$

Using the Cauchy-Schwarz inequality and the Lipschitz continuity of the gradient (10):

$$\begin{aligned} &\mathbb{E} \left[\left(\sum_{j=1}^{\min(k, M-1)} |\omega_j^{(\alpha)}| \|\nabla L(\theta_{k-j})\| \right)^2 \right] \\ &\leq \mathbb{E} \left[\left(\sum_{j=1}^{\min(k, M-1)} |\omega_j^{(\alpha)}|^2 \right) \left(\sum_{j=1}^{\min(k, M-1)} \|\nabla L(\theta_{k-j})\|^2 \right) \right] \\ &= \left(\sum_{j=1}^{\min(k, M-1)} |\omega_j^{(\alpha)}|^2 \right) \sum_{j=1}^{\min(k, M-1)} \mathbb{E}[\|\nabla L(\theta_{k-j})\|^2] \\ &\leq \left(\sum_{j=1}^{\infty} |\omega_j^{(\alpha)}|^2 \right) M \max_{0 \leq i \leq k} \mathbb{E}[\|\nabla L(\theta_i)\|^2]. \end{aligned} \quad (20)$$

Since $\nabla L(\theta) = \nabla L(\theta) - \nabla L(\theta^*) + \nabla L(\theta^*)$, and $\nabla L(\theta^*) = 0$, combining the Lipschitz continuity (10) and bounded noise assumption:

$$\begin{aligned} \mathbb{E}[\|\nabla L(\theta_{k-j})\|^2] &= \mathbb{E}[\|\nabla L(\theta_{k-j}) - \nabla L(\theta^*)\|^2] \\ &\leq L^2 \mathbb{E}[\|\theta_{k-j} - \theta^*\|^2] + \sigma^2. \end{aligned} \quad (21)$$

Defining $\bar{V}_k = \max_{0 \leq i \leq k} \mathbb{E}[\|\theta_i - \theta^*\|^2]$, and selecting $\varepsilon = \mu$, we obtain:

$$\mathbb{E}[(\theta_k - \theta^*)^T G_k^{(\alpha)}]$$

$$\geq \frac{\mu}{2} \mathbb{E}[\|\theta_k - \theta^*\|^2] - \frac{T_\alpha M}{2\mu} (L^2 \bar{V}_k + \sigma^2), \quad (22)$$

For the second moment term of the fractional-order gradient $\mathbb{E}[\|G_k^{(\alpha)}\|^2]$, using the properties of weight coefficients:

$$\begin{aligned} \mathbb{E}[\|G_k^{(\alpha)}\|^2] &= \mathbb{E} \left[\left\| \sum_{j=0}^{\min(k, M-1)} \omega_j^{(\alpha)} \nabla L(\theta_{k-j}) \right\|^2 \right] \\ &\leq \mathbb{E} \left[\left(\sum_{j=0}^{\min(k, M-1)} |\omega_j^{(\alpha)}|^2 \right) \left(\sum_{j=0}^{\min(k, M-1)} \|\nabla L(\theta_{k-j})\|^2 \right) \right] \\ &= \left(\sum_{j=0}^{\min(k, M-1)} |\omega_j^{(\alpha)}|^2 \right) \sum_{j=0}^{\min(k, M-1)} \mathbb{E}[\|\nabla L(\theta_{k-j})\|^2] \\ &\leq S_\alpha M \max_{0 \leq i \leq k} \mathbb{E}[\|\nabla L(\theta_i)\|^2]. \end{aligned} \quad (23)$$

where $S_\alpha = \sum_{j=0}^{\infty} |\omega_j^{(\alpha)}|^2$.

Combining with (21):

$$\mathbb{E}[\|G_k^{(\alpha)}\|^2] \leq S_\alpha M (L^2 \bar{V}_k + \sigma^2). \quad (24)$$

Substituting (22) and (24) into (14):

$$\begin{aligned} V_{k+1} &\leq V_k - \mu \eta V_k + \frac{\eta T_\alpha M L^2}{\mu} \bar{V}_k + \frac{\eta T_\alpha M \sigma^2}{\mu} \\ &\quad + \eta^2 S_\alpha M L^2 \bar{V}_k + \eta^2 S_\alpha M \sigma^2 \\ &= (1 - \mu \eta) V_k + \left(\frac{\eta T_\alpha M L^2}{\mu} + \eta^2 S_\alpha M L^2 \right) \bar{V}_k \\ &\quad + \left(\frac{\eta T_\alpha M \sigma^2}{\mu} + \eta^2 S_\alpha M \sigma^2 \right). \end{aligned} \quad (25)$$

Since $V_k \leq \bar{V}_k$, we have:

$$\begin{aligned} V_{k+1} &\leq \left(1 - \mu \eta + \frac{\eta T_\alpha M L^2}{\mu} + \eta^2 S_\alpha M L^2 \right) \bar{V}_k \\ &\quad + \left(\frac{\eta T_\alpha M \sigma^2}{\mu} + \eta^2 S_\alpha M \sigma^2 \right). \end{aligned} \quad (26)$$

Choosing the learning rate satisfying $\eta < \frac{\mu^2}{L^2 M (2T_\alpha + S_\alpha)}$ ensures:

$$\rho = 1 - \mu \eta + \frac{\eta T_\alpha M L^2}{\mu} + \eta^2 S_\alpha M L^2 < 1. \quad (27)$$

By defining $\bar{V}_k$ and the constant $\Delta = \frac{\eta T_\alpha M \sigma^2}{\mu} + \eta^2 S_\alpha M \sigma^2$, we obtain:

$$\bar{V}_{k+1} \leq \rho \bar{V}_k + \Delta. \quad (28)$$

Recursively solving (28):

$$\bar{V}_k \leq \rho^k \bar{V}_0 + \frac{\Delta}{1 - \rho}. \quad (29)$$

Based on the above derivation, we obtain the theoretical guarantee for the convergence of the FOGD algorithm. According to the recursive relation (29), the FOGD algorithm exhibits linear convergence properties with an error bound of $\bar{V}_k \leq \rho^k \bar{V}_0 + \frac{\Delta}{1 - \rho}$. When the learning rate η satisfies $\eta < \frac{\mu^2}{L^2 M (2T_\alpha + S_\alpha)}$, the convergence factor $\rho < 1$ ensures algorithm convergence. Under small learning rate approximation, the convergence factor can be expressed as $\rho \approx 1 - \eta \left(\mu - \frac{T_\alpha M L^2}{\mu} \right) + O(\eta^2)$, where $O(\cdot)$ denotes the asymptotic upper bound. The dominant term $1 - \mu \eta$ reflects the convergence effect from strong convexity, while the fractional-order memory term $\frac{\eta T_\alpha M L^2}{\mu}$ represents the influence of historical gradient information on convergence speed. The algorithm ultimately converges to a neighborhood centered at the optimal solution with radius $R = \sqrt{\frac{\Delta}{1 - \rho}}$, where Δ is jointly determined by the gradient noise variance σ^2 and learning rate η .

The fractional-order parameter α controls the cumulative effect of historical gradients by adjusting the weight coefficients T_α and S_α .

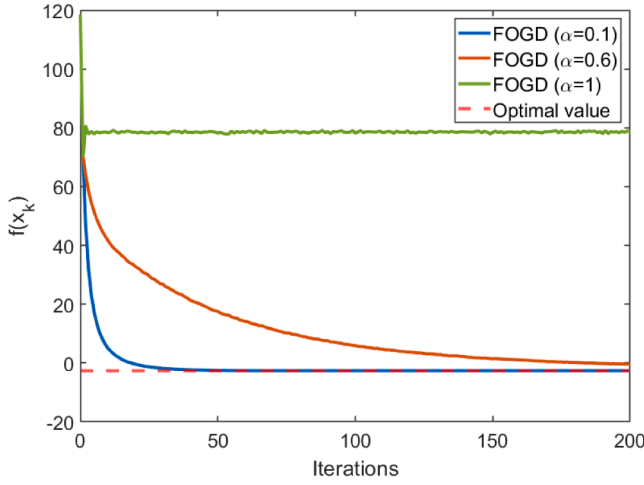


Fig. 1. Convergence on strongly convex function with different orders.

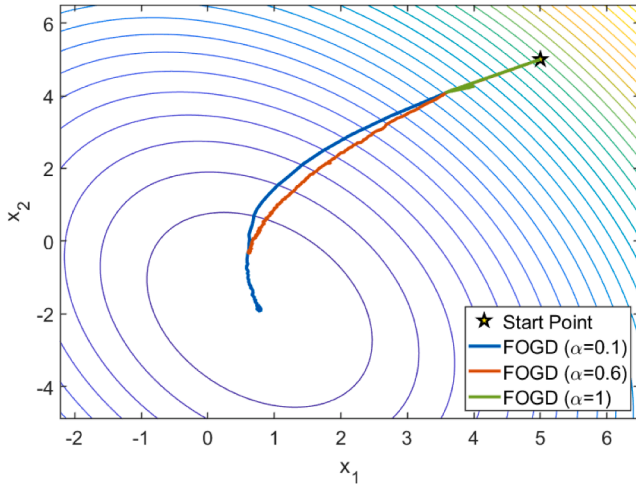


Fig. 2. Optimization trajectories on strongly convex function with different orders.

Smaller α values enhance the algorithm's memory capacity but may slow convergence, requiring a trade-off between memory effects and convergence efficiency. The memory length M directly affects the algorithm's computational complexity and convergence performance: increasing M improves historical information utilization but also increases storage requirements and computational overhead. The learning rate η must balance convergence speed with final accuracy: larger learning rates accelerate initial convergence but increase final error, while smaller learning rates improve final accuracy but reduce convergence speed. Compared to traditional gradient descent, FOGD significantly enhances the algorithm's ability to escape local optima through the non-local memory properties of Grünwald-Letnikov fractional derivatives while maintaining linear convergence characteristics, demonstrating superior stability and global search capabilities when handling optimization problems with complex loss function landscapes.

To validate the theoretical analysis of the FOGD algorithm, we conducted numerical experiments. In the experiments, we selected a two-dimensional strongly convex function as the test case: $f(x) = x_1^2 + 2x_2^2 + 2x_1x_2 + x_1 - 2x_2$ with the global optimum located at $x^* = [0.7778, -1.8889]$, corresponding to the function value $f(x) = -2.6111$. Starting from the initial point $x_0 = [5, 5]$, we set the learning rate $\eta = 0.05$, memory length $M = 20$, and maximum iterations to 200, comparing the effects of different fractional-order parameters on the convergence performance of the FOGD algorithm.

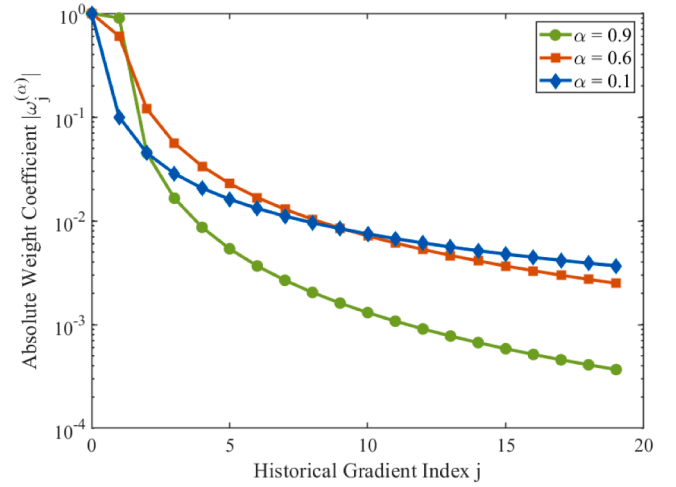


Fig. 3. Weight decay comparison for different α values.

Fig. 1 shows that the FOGD algorithm with three different α values exhibits distinct convergence behaviors. The $\alpha = 0.1$ algorithm rapidly decreases within the first 20 iterations and ultimately converges close to the optimal value; $\alpha = 0.6$ demonstrates a more gradual descent trend but still approaches the optimal solution; while $\alpha = 1.0$ remains near the initial high function value, displaying obvious convergence difficulties and becoming trapped in local optima. This phenomenon visually demonstrates that smaller α values can significantly improve optimization efficiency on strongly convex functions.

Fig. 2 presents the optimization trajectories of the three algorithms in parameter space. Starting from the initial point, the $\alpha = 0.1$ FOGD algorithm rapidly approaches the central region of the contour along a relatively direct path and ultimately stabilizes near the optimal solution; the $\alpha = 0.6$ algorithm moves toward the optimal solution but remains some distance away; while the $\alpha = 1.0$ algorithm hardly leaves the vicinity of the starting point, indicating difficulty in overcoming optimization barriers and reflecting differences in exploration capabilities under different α values.

Fig. 3 displays the decay of weight coefficients $|\omega_j^{(\alpha)}|$ versus historical gradient index j for different α values. In the figure, $\alpha = 0.9$ shows the fastest decay rate; $\alpha = 0.6$ follows; while $\alpha = 0.1$ exhibits the slowest decay trend in the mid-to-late stages. Particularly when $j > 10$, the weight coefficients for $\alpha = 0.1$ are significantly higher than the other cases, meaning long-term historical gradients maintain greater influence in algorithm updates. This observation directly verifies the core idea of the algorithm design: the fractional-order parameter α effectively controls the algorithm's memory strength by regulating the decay rate of weight coefficients $\omega_j^{(\alpha)}$. When α is small, weights decay slowly (as $j \rightarrow \infty$, $\omega_j^{(\alpha)} \sim j^{-\alpha-1}$), increasing the constant C_α and slowing convergence speed, but the algorithm retains more historical gradient information, enhancing global search capabilities and helping escape local optima. When α approaches 1, weights decay rapidly, making C_α smaller and convergence faster, but memory capacity weakens, potentially trapping the algorithm in local optima. Therefore, α should be selected based on the specific optimization problem complexity and loss function landscape characteristics to achieve optimal balance between global search capability and convergence speed.

3. FOGD-based Elman neural network

The Elman recurrent neural network, as a feedforward network with a context layer, processes sequential data through a state feedback mechanism. By incorporating the fractional-order gradient descent algorithm, we can construct an FO-Elman network model with long-term

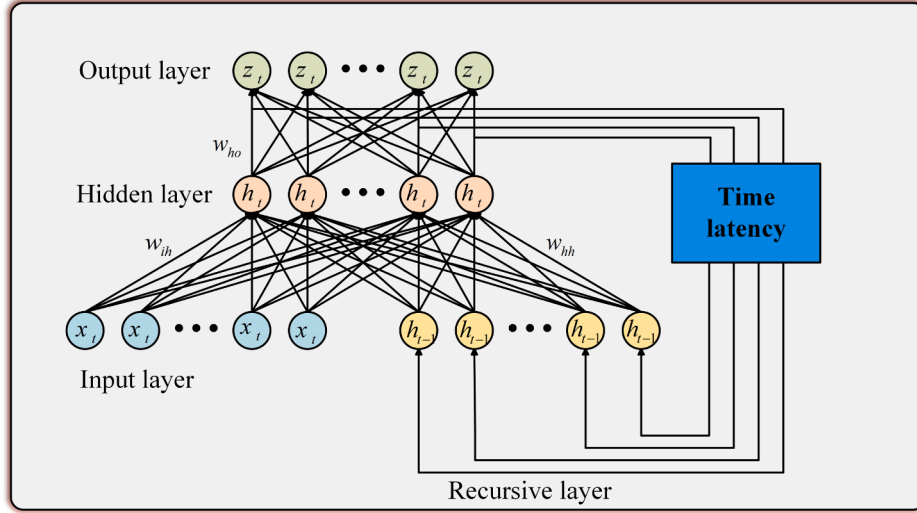
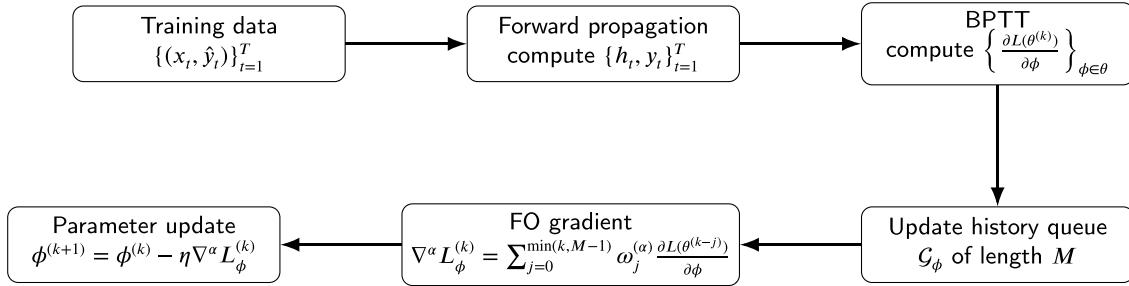


Fig. 4. Topological diagram of Elman neural network.

Fig. 5. Workflow of the FO-Elman training procedure. At iteration k , the standard BPTT computes the current gradients, and the fractional-order update forms $\nabla^\alpha L_\phi^{(k)}$ by aggregating the most recent M stored gradients in $\mathcal{G}_\phi$ with weights $\omega_j^{(\alpha)}$.

memory capabilities. This section will provide a detailed derivation of the complete mathematical framework for this algorithm.

3.1. Algorithm derivation of FO-Elman

The standard Elman network architecture consists of an input layer, hidden layer, context layer, and output layer. The structure of the standard Elman neural network is illustrated in Fig. 4.

Fig. 5 summarizes the key steps of the proposed training strategy. The forward and BPTT recursions over the time index t remain unchanged, while the fractional-order mechanism is applied to the parameter update across optimization iterations by aggregating the gradient history with $\omega_j^{(\alpha)}$ and memory length M .

For an input sequence $\{x_t\}_{t=1}^T$, where $x_t \in \mathbb{R}^{n_i}$, let $h_0 = \mathbf{0}$ be the initial hidden state. The forward propagation process at time step t is described in detail as follows. First, we compute the net input of the hidden layer:

$$z_t^{(h)} = W_{ih}x_t + W_{hh}h_{t-1} + b_h. \quad (30)$$

The hidden state is then calculated through an activation function:

$$h_t = f(z_t^{(h)}) = f(W_{ih}x_t + W_{hh}h_{t-1} + b_h). \quad (31)$$

Next, we compute the input to the output layer:

$$z_t^{(o)} = W_{ho}h_t + b_o. \quad (32)$$

Finally, the network output is obtained:

$$y_t = g(z_t^{(o)}) = g(W_{ho}h_t + b_o), \quad (33)$$

where $h_t \in \mathbb{R}^{n_h}$ is the hidden state vector at time t , $y_t \in \mathbb{R}^{n_o}$ is the network output vector, $W_{ih} \in \mathbb{R}^{n_h \times n_i}$ is the weight matrix from input to

hidden layer, $W_{hh} \in \mathbb{R}^{n_h \times n_h}$ is the recurrent weight matrix of the hidden layer, $W_{ho} \in \mathbb{R}^{n_o \times n_h}$ is the weight matrix from hidden to output layer, $b_h \in \mathbb{R}^{n_h}$ and $b_o \in \mathbb{R}^{n_o}$ are the bias vectors for the hidden and output layers respectively, $f(\cdot)$ is the hidden layer activation function, and $g(\cdot)$ is the output layer activation function. The network parameter set is defined as:

$$\theta = \{W_{ih}, W_{hh}, W_{ho}, b_h, b_o\}. \quad (34)$$

For an input sequence of length T , we define the mean squared error loss function:

$$L(\theta) = \frac{1}{2T} \sum_{t=1}^T \|y_t - \hat{y}_t\|^2 = \frac{1}{2T} \sum_{t=1}^T \sum_{i=1}^{n_o} (y_t^{(i)} - \hat{y}_t^{(i)})^2, \quad (35)$$

where $\hat{y}_t$ is the target output.

To apply fractional-order gradient descent to the Elman neural network, we first derive the conventional gradient computation formulas. Using the backpropagation through time (BPTT) algorithm, the gradient computation for each parameter proceeds as follows. For the output layer error term:

$$\delta_t^{(o)} = \frac{\partial L}{\partial z_t^{(o)}} = \frac{\partial L}{\partial y_t} \odot g'(z_t^{(o)}), \quad (36)$$

where $\odot$ denotes element-wise multiplication, and $g'(\cdot)$ is the derivative of the output activation function. The hidden layer error term must account for error propagation from both the output layer and the next time step's hidden layer:

$$\delta_t^{(h)} = \frac{\partial L}{\partial z_t^{(h)}} = (W_{ho}^T \delta_t^{(o)} + W_{hh}^T \delta_{t+1}^{(h)}) \odot f'(z_t^{(h)}). \quad (37)$$

Remark 3. The index t denotes the time step in a sequence for BPTT, while the index k denotes the optimization iteration for parameter updates. Throughout this paper, the fractional-order operation is applied along the optimization iterations rather than along the temporal recursion.

In the FO-Elman framework, each parameter update depends not only on the current gradient but also on historical gradient information. Let the current iteration be k . For any parameter $\phi \in \theta$, its fractional-order gradient is defined as:

$$\nabla^\alpha L_\phi^{(k)} = \sum_{j=0}^{\min(k, M-1)} \omega_j^{(\alpha)} \frac{\partial L(\theta_{k-j})}{\partial \phi}, \quad (38)$$

where the weight coefficients $\omega_j^{(\alpha)}$ are as described in the previous formula (6).

Thus, we obtain the fractional-order gradients for each weight:

For the output layer weight W_{ho} :

$$\begin{aligned} \nabla^\alpha L_{W_{ho}}^{(k)} &= \sum_{j=0}^{\min(k, M-1)} \omega_j^{(\alpha)} \left[\sum_{t=1}^T \delta_t^{(o), (k-j)} h_t^{T, (k-j)} \right] \\ &= \sum_{j=0}^{\min(k, M-1)} \omega_j^{(\alpha)} \left[\sum_{t=1}^T \left(\frac{\partial L(\theta_{k-j})}{\partial y_t^{(k-j)}} \odot g'(z_t^{(o), (k-j)}) \right) \right. \\ &\quad \times f^T(W_{ih}^{(k-j)} x_t + W_{hh}^{(k-j)} h_{t-1}^{(k-j)} + b_h^{(k-j)}) \left. \right]. \end{aligned} \quad (39)$$

For the input layer weight W_{ih} :

$$\begin{aligned} \nabla^\alpha L_{W_{ih}}^{(k)} &= \sum_{j=0}^{\min(k, M-1)} \omega_j^{(\alpha)} \left[\sum_{t=1}^T \delta_t^{(h), (k-j)} x_t^{T, (k-j)} \right] \\ &= \sum_{j=0}^{\min(k, M-1)} \omega_j^{(\alpha)} \left[\sum_{t=1}^T \left(\left(W_{ho}^{T, (k-j)} \delta_t^{(o), (k-j)} \right. \right. \right. \\ &\quad \left. \left. + W_{hh}^{T, (k-j)} \delta_{t+1}^{(h), (k-j)} \right) \odot f'(W_{ih}^{(k-j)} x_t + W_{hh}^{(k-j)} h_{t-1}^{(k-j)} + b_h^{(k-j)}) \right) x_t^{T, (k-j)} \right]. \end{aligned} \quad (40)$$

For the recurrent weight W_{hh} :

$$\begin{aligned} \nabla^\alpha L_{W_{hh}}^{(k)} &= \sum_{j=0}^{\min(k, M-1)} \omega_j^{(\alpha)} \left[\sum_{t=2}^T \delta_t^{(h), (k-j)} h_{t-1}^{T, (k-j)} \right] \\ &= \sum_{j=0}^{\min(k, M-1)} \omega_j^{(\alpha)} \left[\sum_{t=2}^T \left(W_{ho}^{T, (k-j)} \delta_t^{(o), (k-j)} \right. \right. \\ &\quad \left. \left. + W_{hh}^{T, (k-j)} \delta_{t+1}^{(h), (k-j)} \right) \odot f'(W_{ih}^{(k-j)} x_t + W_{hh}^{(k-j)} h_{t-1}^{(k-j)} + b_h^{(k-j)}) h_{t-1}^{T, (k-j)} \right]. \end{aligned} \quad (41)$$

For the output layer bias b_o :

$$\begin{aligned} \nabla^\alpha L_{b_o}^{(k)} &= \sum_{j=0}^{\min(k, M-1)} \omega_j^{(\alpha)} \left[\sum_{t=1}^T \delta_t^{(o), (k-j)} \right] \\ &= \sum_{j=0}^{\min(k, M-1)} \omega_j^{(\alpha)} \left[\sum_{t=1}^T \frac{\partial L(\theta_{k-j})}{\partial y_t^{(k-j)}} \odot g'(z_t^{(o), (k-j)}) \right]. \end{aligned} \quad (42)$$

For the hidden layer bias b_h :

$$\begin{aligned} \nabla^\alpha L_{b_h}^{(k)} &= \sum_{j=0}^{\min(k, M-1)} \omega_j^{(\alpha)} \left[\sum_{t=1}^T \delta_t^{(h), (k-j)} \right] \\ &= \sum_{j=0}^{\min(k, M-1)} \omega_j^{(\alpha)} \left[\sum_{t=1}^T \left(W_{ho}^{T, (k-j)} \delta_t^{(o), (k-j)} \right. \right. \\ &\quad \left. \left. + W_{hh}^{T, (k-j)} \delta_{t+1}^{(h), (k-j)} \right) \odot f'(W_{ih}^{(k-j)} x_t + W_{hh}^{(k-j)} h_{t-1}^{(k-j)} + b_h^{(k-j)}) \right] \end{aligned} \quad (43)$$

where the superscript $(k-j)$ denotes the corresponding quantity at the $(k-j)$ th iteration. Based on the fractional-order gradients, the parameter update rules for the FO-Elman algorithm are:

$$W_{ho}^{(k+1)} = W_{ho}^{(k)} - \eta \nabla^\alpha L_{W_{ho}}^{(k)}, \quad (44)$$

$$W_{ih}^{(k+1)} = W_{ih}^{(k)} - \eta \nabla^\alpha L_{W_{ih}}^{(k)}, \quad (45)$$

$$W_{hh}^{(k+1)} = W_{hh}^{(k)} - \eta \nabla^\alpha L_{W_{hh}}^{(k)}, \quad (46)$$

$$b_o^{(k+1)} = b_o^{(k)} - \eta \nabla^\alpha L_{b_o}^{(k)}, \quad (47)$$

$$b_h^{(k+1)} = b_h^{(k)} - \eta \nabla^\alpha L_{b_h}^{(k)}. \quad (48)$$

The FO-Elman algorithm requires storing historical gradient information. For each parameter ϕ , we maintain a gradient history queue of length M :

$$G_\phi = \left\{ \frac{\partial L(\theta_k)}{\partial \phi}, \frac{\partial L(\theta_{k-1})}{\partial \phi}, \dots, \frac{\partial L(\theta_{k-M+1})}{\partial \phi} \right\}. \quad (49)$$

Remark 4. At iteration k , we compute the standard BPTT gradient $g_\phi^{(k)}$ for each parameter ϕ . We then update the gradient history queue and form the fractional-order gradient by a weighted sum of the most recent M stored gradients, which is directly used in the parameter update rule.

The total storage complexity is $O(M \cdot |\theta|)$, where $|\theta|$ is the total number of parameters:

$$|\theta| = n_h \cdot n_i + n_h \cdot n_h + n_o \cdot n_h + n_h + n_o. \quad (50)$$

In terms of computational complexity, the time complexity for each parameter update is $O(M \cdot |\theta|)$, which is M times higher than the $O(|\theta|)$ of conventional gradient descent algorithms.

The convergence of the FO-Elman algorithm is influenced by the fractional-order parameter α and memory length M . The decay characteristics of the memory weights ensure algorithm stability:

$$\lim_{j \rightarrow \infty} \omega_j^{(\alpha)} = 0, \quad \forall \alpha \in (0, 1). \quad (51)$$

Additionally, the weight coefficients satisfy a modified normalization condition:

$$\sum_{j=0}^{\infty} \omega_j^{(\alpha)} = 0, \quad \alpha \in (0, 1). \quad (52)$$

However, $\omega_0^{(\alpha)} = 1$ ensures that the current gradient always contributes to the update. For loss functions satisfying the Lipschitz continuity condition and the strong convexity assumption, we can prove that the FO-Elman algorithm admits convergence guarantees when an appropriate learning rate η and fractional-order parameter α are chosen. The convergence behavior is closely related to α : when $\alpha \rightarrow 0$, the weights on historical gradients vanish and the update reduces to conventional gradient descent using only the current gradient, yielding faster and more stable convergence but limited memory. In contrast, when $\alpha \rightarrow 1$, the update approaches a gradient-difference-type correction dominated by the most recent two gradients, which effectively increases temporal coupling but may introduce oscillations and slow down convergence. The pseudocode of the algorithm 1 is as follows:

4. Experimental results

The Elman neural network, as a variant of recurrent neural networks (RNNs), is commonly employed in temporal modeling tasks and time series prediction problems. To evaluate the effectiveness of the proposed method in the field of system identification, we consider two scenarios: (1) identification of a highly nonlinear abstract system, and (2) identification of a liquid level control system comprising two cascaded water tanks. In these two system identification tasks, fractional-order gradient descent (FOGD) with different fractional orders is compared with

Algorithm 1 FOGD for Elman neural network (FO-Elman).

Require: Training data $\{(x_t, \hat{y}_t)\}_{t=1}^T$, learning rate η , fractional order $\alpha \in (0, 1)$, memory length M

Ensure: Trained Elman network parameters $\theta = \{W_{ih}, W_{hh}, W_{ho}, b_h, b_o\}$

```

1: Initialize network parameters  $\theta^{(0)}$ 
2: Initialize gradient history queues  $\mathcal{G}_\phi$  for all  $\phi \in \theta$ 
3: Compute fractional weights  $\omega_j^{(\alpha)} = (-1)^j \frac{\Gamma(\alpha+1)}{\Gamma(j+1)\Gamma(\alpha-j+1)}$  for
    $j = 0, 1, \dots, M-1$ 
4:  $k = 0$ 
5: while not converged do
6:   Set initial hidden state  $h_0 = 0$ 
7:   // Forward propagation
8:   for  $t = 1$  to  $T$  do
9:      $z_t^{(h)} = W_{ih}^{(k)} x_t + W_{hh}^{(k)} h_{t-1} + b_h^{(k)}$ 
10:     $h_t = f(z_t^{(h)})$ 
11:     $z_t^{(o)} = W_{ho}^{(k)} h_t + b_o^{(k)}$ 
12:     $y_t = g(z_t^{(o)})$ 
13:   end for
14:   Compute loss  $L(\theta^{(k)}) = \frac{1}{2T} \sum_{t=1}^T \|y_t - \hat{y}_t\|^2$ 
15:   // Backward propagation using BPTT
16:   Compute current gradients  $\nabla L(\theta^{(k)}) = \{\frac{\partial L}{\partial \phi}\}_{\phi \in \theta}$ 
17:   for each parameter  $\phi \in \theta$  do
18:     Update gradient history queue  $\mathcal{G}_\phi$  with current gradient  $\frac{\partial L(\theta^{(k)})}{\partial \phi}$ 
19:     Compute fractional gradient:  $\nabla^\alpha L_\phi^{(k)} = \sum_{j=0}^{\min(k, M-1)} \omega_j^{(\alpha)} \frac{\partial L(\theta_{k-j}^{(k)})}{\partial \phi}$ 
20:     Update parameter:  $\phi^{(k+1)} = \phi^{(k)} - \eta \nabla^\alpha L_\phi^{(k)}$ 
21:   end for
22:    $k = k + 1$ 
23: end while

```

traditional gradient descent (GD) as well as several widely used optimization algorithms, including stochastic gradient descent with momentum (SGDM), RMSProp, and Adam (Algorithm 1). In the domain of time series prediction, we also examine two cases: (1) prediction of the Mackey-Glass time series, and (2) real-world electricity load data from New South Wales, Australia. In these two prediction tasks, Elman neural networks with different fractional orders are compared with the traditional Elman network, as well as LSTM and GRU models trained using the Adam optimizer. All simulation experiments were conducted on an Intel(R) Core(TM) i5-14400 2.50 GHz machine with 32 GB RAM using MATLAB R2023b, with each simulation repeated ten times to obtain averaged results. All hyperparameters in the experiments were selected via grid search on the training set. The learning rate was chosen from the set $\{0.01, 0.005, 0.001, 0.0005, 0.0001\}$, and the number of hidden neurons was selected from $\{8, 16, 32, 64\}$. The reported results correspond to the best-performing configurations under these settings. For LSTM and GRU baselines, the Adam optimizer was employed with default settings except for the learning rate, in order to ensure a fair and reproducible comparison.

4.1. System identification

4.1.1. Nonlinear system identification

A highly nonlinear system is described by the following equation (Khan et al., 2018):

$$y(t) = y(t-1) - 0.25y(t-2) + 0.1045 \left[\cos(2u(t)) + \exp(-10|u(t)|) \right] + 0.0902u(t-1) + n(t), \quad (53)$$

where $u(t)$ and $y(t)$ represent the system input and output respectively, and $n(t)$ denotes zero-mean Gaussian noise with $n(0, 0.0002)$. In simulations, the input signal $u(t)$ is configured as a step function maintaining

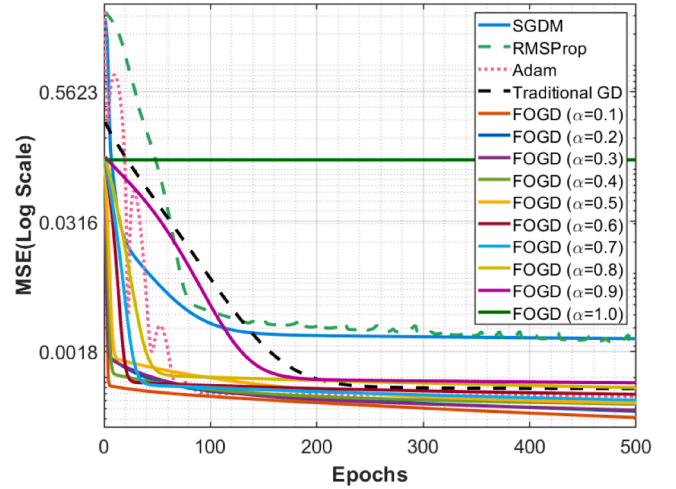


Fig. 6. Highly nonlinear system identification: Iteration curves of different fractional orders versus conventional optimization methods.

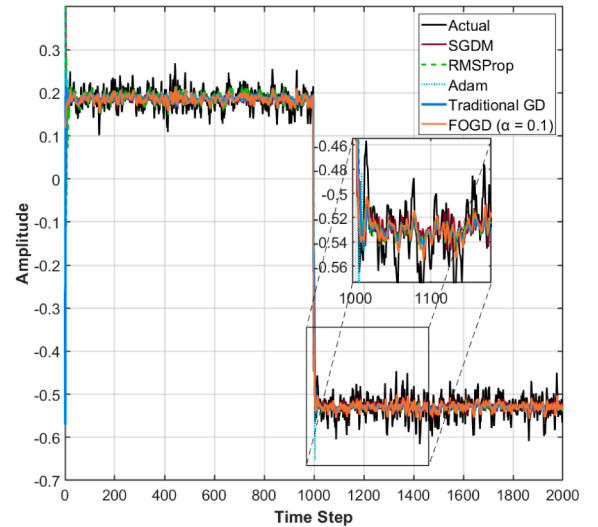


Fig. 7. Output of nonlinear system identification.

unit amplitude of 1 in the first half-cycle and -1 in the second half-cycle. The Elman neural network architecture consists of an input layer, a hidden layer, and an output layer. The input layer comprises three nodes: the current input $u(t)$ and two delayed outputs $y(t-1)$ and $y(t-2)$. The hidden layer contains 16 neurons, while the output layer has a single neuron. The network is trained using a fractional-order gradient descent algorithm based on GL definition, with learning rate η set to 0.01. Comparative experiments were conducted for fractional orders α ranging from 0.1 to 1.0 in increments of 0.1, alongside traditional gradient descent. The fractional-order memory length M was fixed at 10, with 500 training epochs. To ensure reproducibility, the random seed was fixed at 1.

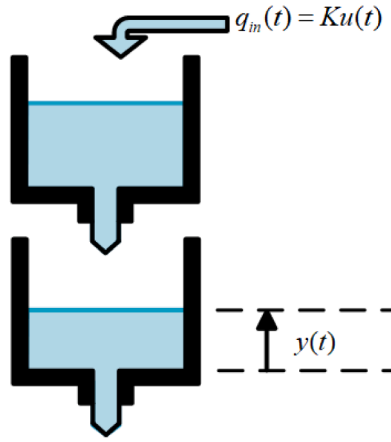
Fig. 6 presents the MSE convergence curves of FOGD with different fractional orders in comparison with traditional GD, SGDM, RMSProp, and Adam over 500 epochs. Most fractional-order configurations exhibit faster convergence and lower steady-state MSE than GD, among which FOGD with $\alpha = 0.1$ achieves the best performance in terms of both convergence speed and final MSE.

Compared with SGDM, RMSProp, and Adam, FOGD demonstrates more stable convergence behavior and improved asymptotic accuracy across most fractional orders. Although adaptive methods such as RMSProp and Adam accelerate the initial convergence, they either con-

Table 1

Performance comparison of different optimization methods for the nonlinear system identification task.

Method	MSE	Training Time (s)	Model Size
Traditional GD–Elman	7.81e-04	1.63	301
SGDM	2.356e-03	1.00	301
RMSProp	2.161e-03	1.14	301
Adam	6.57e-04	9.91	301
FO–Elman ($\alpha = 0.1$)	4.13e-04	4.87	301
FO–Elman ($\alpha = 0.2$)	4.70e-04	4.68	301
FO–Elman ($\alpha = 0.3$)	4.77e-04	4.62	301
FO–Elman ($\alpha = 0.4$)	5.51e-04	4.63	301
FO–Elman ($\alpha = 0.5$)	5.50e-04	4.60	301
FO–Elman ($\alpha = 0.6$)	6.64e-04	4.59	301
FO–Elman ($\alpha = 0.7$)	4.79e-04	4.62	301
FO–Elman ($\alpha = 0.8$)	6.11e-04	4.64	301
FO–Elman ($\alpha = 0.9$)	5.19e-04	4.62	301
FO–Elman ($\alpha = 1.0$)	1.27268e-01	4.61	301

**Fig. 8.** Schematic diagram.

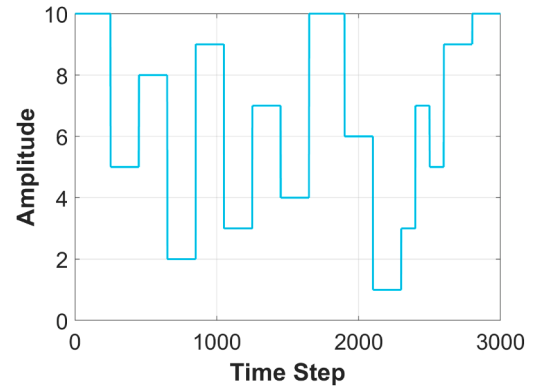
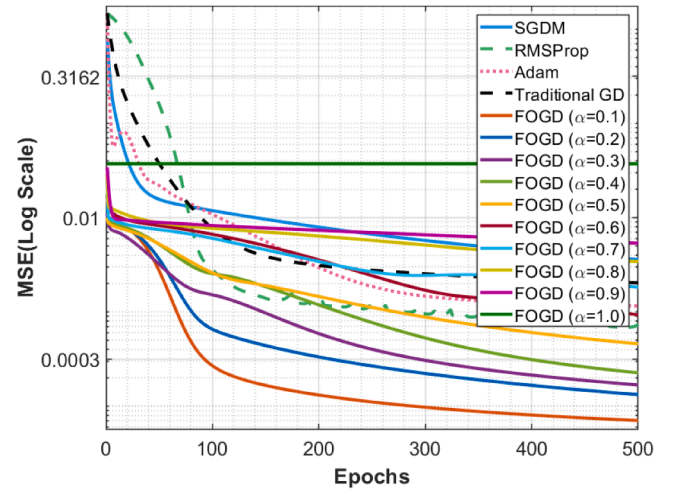
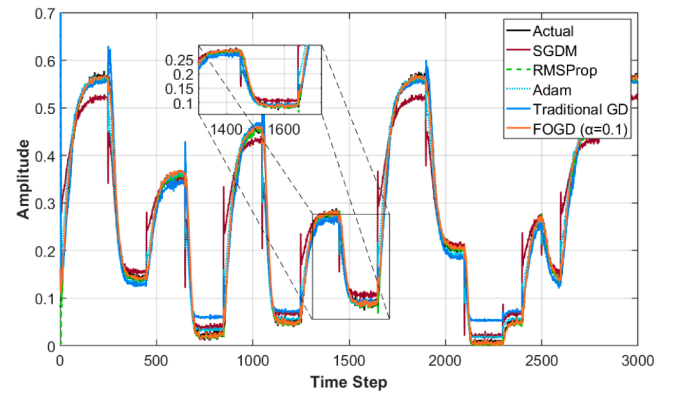
verge to higher MSE values or require substantially longer training time. In contrast, FOGD with appropriately selected fractional orders (e.g., $\alpha = 0.1$ – 0.3) achieves a favorable balance between convergence accuracy and computational efficiency.

Fig. 7 further illustrates that the output response obtained by FOGD ($\alpha = 0.1$) exhibits closer agreement with the target values than that of traditional GD. These qualitative observations are consistent with the quantitative results reported in Table 1. Since all methods are evaluated under identical network architectures with the same number of parameters, the observed performance improvements can be attributed to the optimization strategy rather than model capacity.

4.1.2. Dual-tank system identification

This subsection examines a black-box dynamic model of a nonlinear dual-tank system (The MathWorks, 2024). The input $u(t)$ represents the voltage [V] applied to an electric pump, which generates inflow $q_{in}(t)$ to the upper tank. Water flows through an orifice at the bottom of the upper tank to fill the lower tank. The system output $y(t)$ corresponds to the liquid level [m] in the lower tank. The dataset contains 3000 input-output measurements sampled at 0.2-s intervals. The schematic diagram and input signal of the dual-tank system are shown in Figs. 8 and 9. The Elman network configuration remains identical to the previous subsection.

Fig. 10 illustrates the MSE convergence behavior of different optimization methods for the dual-tank system identification task. The results in Fig. 10 indicate that FOGD with small fractional orders achieves significantly faster convergence and lower steady-state MSE than traditional GD.

**Fig. 9.** Input signal of dual-tank system.**Fig. 10.** Dual-tank system identification: Iteration curves of different fractional orders versus conventional optimization methods.**Fig. 11.** Output of dual-tank system identification.

As summarized in Table 2, FOGD with $\alpha = 0.1$ attains the smallest MSE of 6.00×10^{-5} , which is substantially lower than those obtained by GD, SGDM, RMSProp, and Adam. Although RMSProp achieves relatively low MSE with shorter training time, its final accuracy remains inferior to that of the optimal fractional-order configurations. Adam, on the other hand, incurs a considerably higher computational cost while failing to achieve comparable accuracy.

Fig. 11 further shows that the output identified by FOGD ($\alpha = 0.1$) exhibits closer agreement with the true system response than that obtained by GD. These results indicate that the proposed fractional-order

Table 2
Performance comparison of different optimization methods for the dual-tank system identification task.

Method	MSE	Training Time (s)	Model Size
Traditional GD–Elman	2.033e-03	1.56	301
SGDM	3.591e-03	1.45	301
RMSProp	6.86e-04	1.45	301
Adam	1.153e-03	13.80	301
FO–Elman ($\alpha = 0.1$)	6.00e-05	7.13	301
FO–Elman ($\alpha = 0.2$)	1.56e-04	7.01	301
FO–Elman ($\alpha = 0.3$)	2.16e-04	7.11	301
FO–Elman ($\alpha = 0.4$)	1.35e-04	7.09	301
FO–Elman ($\alpha = 0.5$)	3.09e-04	6.99	301
FO–Elman ($\alpha = 0.6$)	4.54e-04	7.07	301
FO–Elman ($\alpha = 0.7$)	1.141e-03	6.99	301
FO–Elman ($\alpha = 0.8$)	5.796e-03	7.05	301
FO–Elman ($\alpha = 0.9$)	8.607e-03	7.04	301
FO–Elman ($\alpha = 1.0$)	3.273e-02	7.09	301

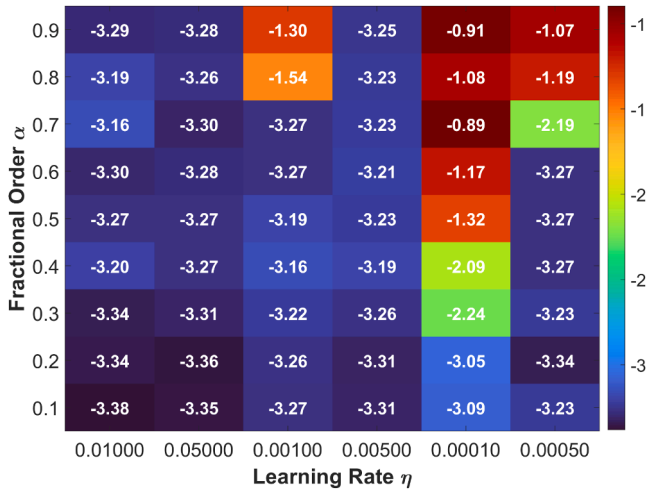


Fig. 12. Heatmap of $\log_{10}(\text{MSE})$ with respect to the fractional order α and learning rate η .

optimization approach provides enhanced robustness and accuracy for dual-tank system identification.

4.2. Parameter sensitivity analysis

To further assist readers in understanding the influence of key hyperparameters on the proposed fractional-order optimization method, a parameter sensitivity analysis is conducted using the nonlinear system identification task described in Section 4.1.1. All results reported in this subsection are obtained after 300 training epochs in order to focus on the steady-state behavior of the optimization process.

Fig. 12 illustrates the joint effect of the fractional order α and learning rate η on the final MSE. It can be observed that stable low-error regions are formed for small fractional orders ($\alpha \leq 0.3$) across a relatively wide range of learning rates. In contrast, larger fractional orders exhibit increased sensitivity to the learning rate and lead to degraded performance, particularly when η is either too small or too large. This indicates that smaller fractional orders provide increased tolerance to learning-rate selection.

The influence of the memory length M is investigated in Fig. 13, which presents the MSE heatmap with respect to α and M . The results indicate that moderate memory lengths (approximately $M = 10$ – 20) achieve consistently low MSE across a wide range of fractional orders. In contrast, excessively large memory lengths do not yield further performance improvements and may even degrade accuracy for certain fractional orders.

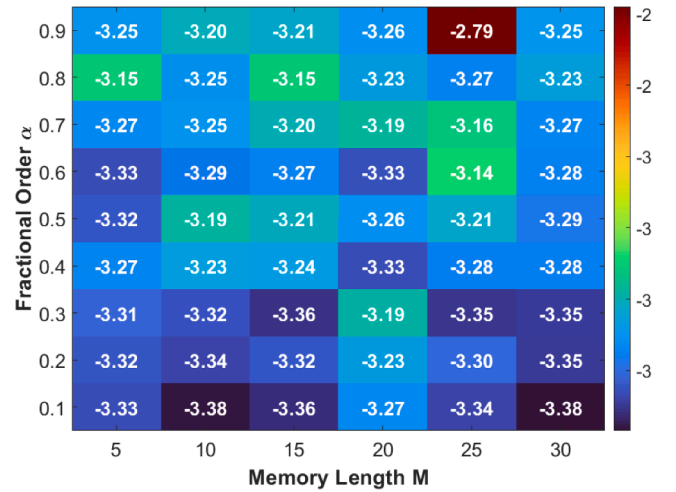


Fig. 13. Heatmap of $\log_{10}(\text{MSE})$ as a function of the fractional order α and memory length M .

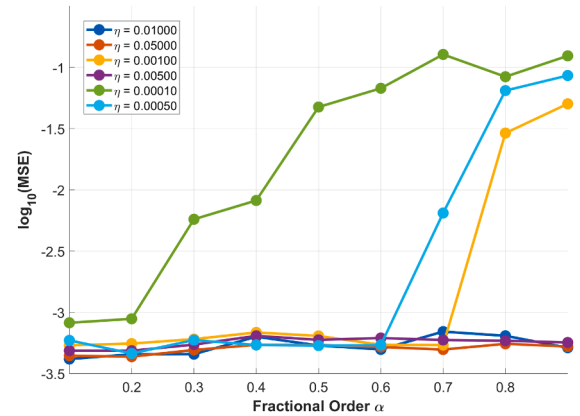


Fig. 14. Variation of $\log_{10}(\text{MSE})$ with respect to the fractional order α under different learning rates η .

Fig. 14 further examines the relationship between α and MSE under different learning rates when the memory length is fixed at $M = 10$. For moderate learning rates (e.g., $\eta = 0.01$ and $\eta = 0.005$), the performance remains relatively stable for α in the range of approximately 0.1 to 0.3. However, for extremely small learning rates, the performance deteriorates rapidly as α increases, highlighting the interaction between fractional order and step size in fractional-order optimization.

From a computational perspective, Fig. 15 shows the training time as a function of α and M . While the training time increases monotonically with the memory length M , it is only weakly affected by the fractional order α . This suggests that M is the primary factor governing computational cost, whereas α mainly influences convergence accuracy rather than efficiency.

Remark 5. Based on the above analysis, selecting the fractional order within $\alpha \in [0.1, 0.3]$ and the memory length around $M = 10$ leads to reliable and well-behaved optimization performance. These parameter settings achieve a balanced trade-off among convergence accuracy, sensitivity to learning-rate selection, and computational efficiency.

4.3. Time series prediction

4.3.1. Mackey-glass time series prediction

This section examines the Mackey-Glass time series prediction problem (López-Caraballo et al., 2016), described by the following delay



Fig. 15. Training time (in seconds) as a function of the fractional order α and memory length M .

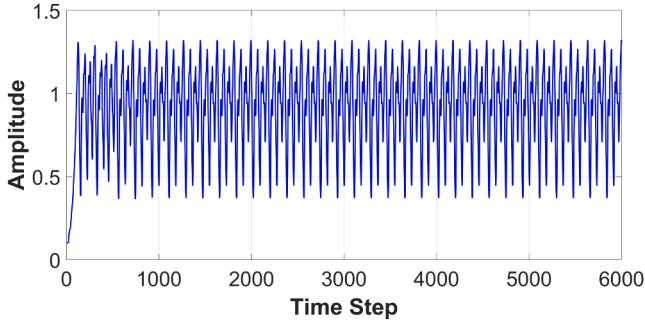


Fig. 16. Mackey-glass time series.

differential equation:

$$\frac{dx(t)}{dt} = \frac{ax(t-\tau)}{1+x(t-\tau)^{10}} - bx(t), \quad (54)$$

where for $0 \leq \tau \leq t$, parameters are set as $b = 0.1$, $a = 0.2$, $\tau = 20$, with initial condition $x(t-\tau) = 0$. The time series $x(t)$ is generated from Eq. (54) for $t = 1, 2, \dots, 6000$.

The dataset was generated by sampling the curve $x(t)$ at 1-s intervals, yielding 6000 samples divided into training and test sets at a 7:3 ratio. The Elman network architecture includes 8 input neurons (current input, 3 delayed inputs, and 4 delayed outputs), 16 hidden neurons, and 1 output neuron. The output layer employs a linear activation function, while other layers use sigmoid activation. For LSTM and GRU models, a single hidden layer with 16 hidden units was employed in this experiment. Training parameters include: learning rate 0.01, fractional-order memory length 10, and 500 training epochs. Fig. 16 shows the Mackey-Glass time series.

Fig. 17 presents the MSE convergence curves of FOGD with different fractional orders for the Mackey–Glass time series prediction task, together with traditional GD and recurrent neural network baselines. It can be observed that fractional-order optimization significantly improves the training dynamics of the Elman network, yielding faster error reduction and lower final training MSE for appropriate fractional orders. Among all configurations, FOGD with $\alpha = 0.2$ achieves the lowest training error, indicating its effectiveness in capturing the complex temporal dependencies of the Mackey–Glass system.

Fig. 18 compares the prediction results on the test set obtained by different models. As summarized in Table 3, the FO–Elman model with $\alpha = 0.2$ achieves the smallest test MSE while maintaining a compact network structure and moderate training cost. Although LSTM and GRU

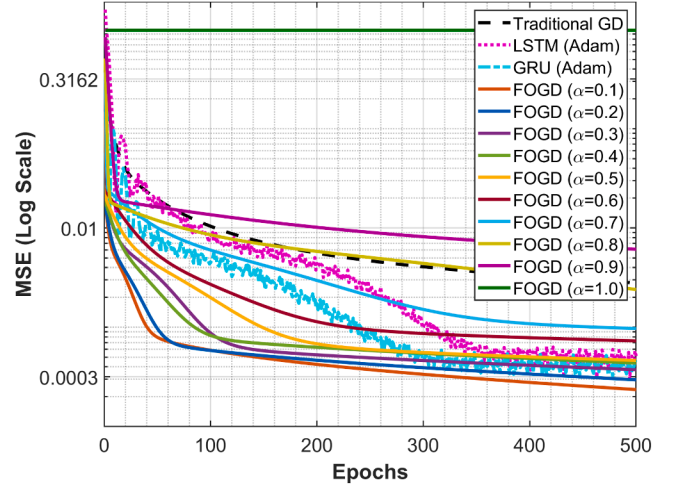


Fig. 17. Mackey-glass time series prediction: Iteration curves of different fractional orders compared with multiple methods.

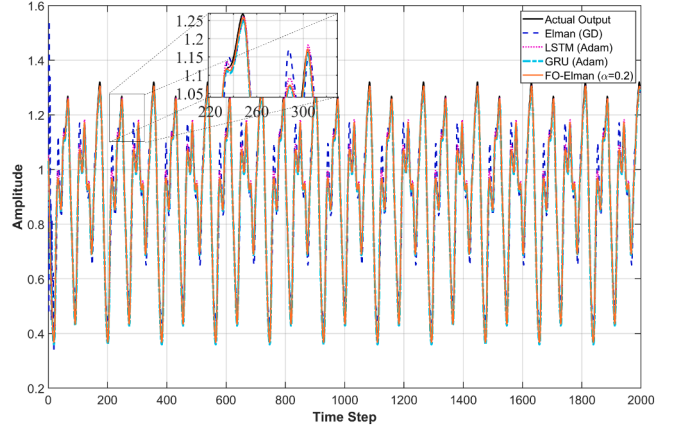


Fig. 18. Test set results of Mackey-glass time series prediction.

models trained with Adam also attain competitive prediction accuracy, they require substantially larger model sizes and significantly longer training time. These results suggest that fractional-order optimization enables the Elman network to achieve high prediction accuracy with improved efficiency, providing an effective alternative to more complex recurrent architectures for chaotic time-series prediction.

4.3.2. Electricity load time series prediction

This experiment utilizes actual electricity load data from New South Wales, Australia (2006–2010), sampled at 30-min intervals (87,648 samples over 3652 days). The dataset is partitioned into training and test sets at an 8:2 ratio. The Elman network architecture incorporates 24 input neurons (representing 12-h historical load data), 16 hidden neurons, and 1 output neuron for single-step prediction. Other hyperparameters remain consistent with previous configurations. For LSTM and GRU models, a single hidden layer with 16 hidden units was used. As shown in Fig. 19, the original load data exhibits significant periodicity and complex fluctuation patterns, posing substantial challenges for time series forecasting models.

Fig. 20 presents the training MSE convergence curves of different models for the long-horizon load time-series prediction task. It can be observed that traditional GD exhibits poor convergence behavior and remains at a high error level throughout training. In contrast, fractional-order optimization significantly improves the learning dynamics of the Elman network, leading to substantially lower training MSE across a wide range of fractional orders. Although LSTM and GRU trained with

Table 3

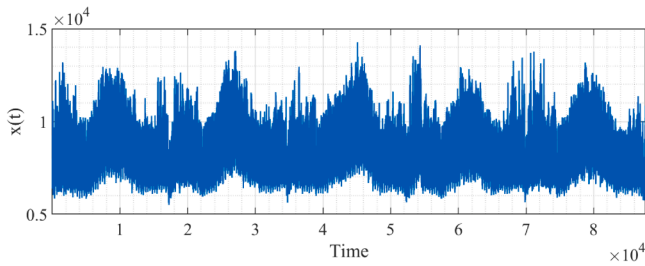
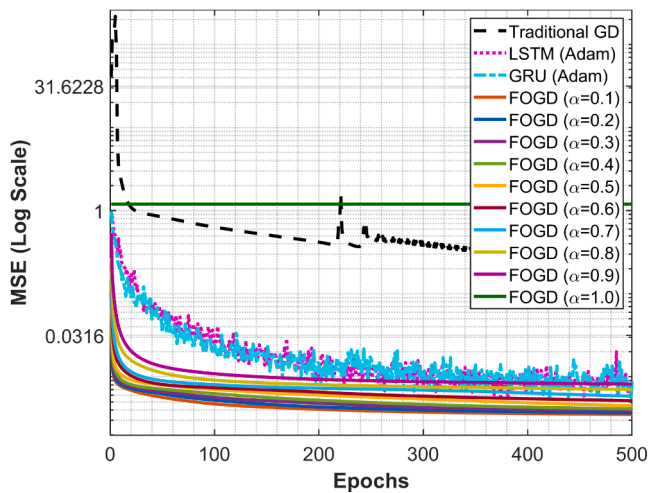
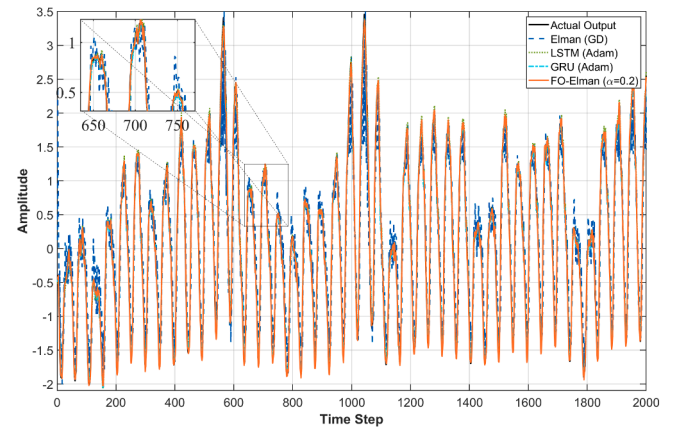
Performance comparison of different models for Mackey–Glass time series prediction.

Model	Training MSE	Test MSE	Training Time (s)	Model Size
Traditional GD–Elman	2.788e-03	3.425e-03	1.93	376
LSTM (Adam)	2.19e-04	2.93e-04	45.64	1456
GRU (Adam)	2.10e-04	2.14e-04	48.36	1096
FO–Elman ($\alpha = 0.1$)	2.19e-04	2.22e-04	9.60	376
FO–Elman ($\alpha = 0.2$)	1.12e-04	1.06e-04	9.47	376
FO–Elman ($\alpha = 0.3$)	3.65e-04	3.68e-04	9.43	376
FO–Elman ($\alpha = 0.4$)	4.27e-04	4.32e-04	9.50	376
FO–Elman ($\alpha = 0.5$)	4.47e-04	4.54e-04	9.46	376
FO–Elman ($\alpha = 0.6$)	6.96e-04	7.07e-04	9.43	376
FO–Elman ($\alpha = 0.7$)	9.49e-04	9.67e-04	9.46	376
FO–Elman ($\alpha = 0.8$)	2.059e-03	2.120e-03	9.42	376
FO–Elman ($\alpha = 0.9$)	5.585e-03	5.745e-03	9.48	376
FO–Elman ($\alpha = 1.0$)	8.881e-01	9.026e-01	9.44	376

Table 4

Performance comparison of different models for long-horizon load time-series prediction.

Model	Training MSE	Test MSE	Training Time (s)	Model Size
Traditional GD–Elman	1.617e-01	1.534e-01	29.93	616
LSTM (Adam)	5.712e-03	5.420e-03	806.00	2416
GRU (Adam)	5.103e-03	4.695e-03	810.72	1816
FO–Elman ($\alpha = 0.1$)	5.153e-03	4.481e-03	182.78	616
FO–Elman ($\alpha = 0.2$)	4.627e-03	4.443e-03	184.17	616
FO–Elman ($\alpha = 0.3$)	5.113e-03	4.711e-03	182.57	616
FO–Elman ($\alpha = 0.4$)	5.089e-03	4.598e-03	183.65	616
FO–Elman ($\alpha = 0.5$)	5.101e-03	4.671e-03	182.91	616
FO–Elman ($\alpha = 0.6$)	5.908e-03	5.269e-03	183.68	616
FO–Elman ($\alpha = 0.7$)	6.086e-03	5.435e-03	185.47	616
FO–Elman ($\alpha = 0.8$)	6.916e-03	6.233e-03	185.69	616
FO–Elman ($\alpha = 0.9$)	7.621e-03	6.819e-03	184.79	616
FO–Elman ($\alpha = 1.0$)	1.204e+00	1.137e+00	185.91	616

**Fig. 19.** Electricity load data.**Fig. 20.** Iteration curves of different fractional orders compared with multiple methods.**Fig. 21.** Test set results of electricity load single-step prediction.

Adam also achieve effective error reduction, their convergence exhibits noticeable fluctuations and requires considerably longer training time.

Fig. 21 compares the prediction results on the test set. As summarized in Table 4, FO–Elman models achieve test MSE values that are comparable to those of LSTM and GRU while employing significantly fewer parameters and substantially lower computational cost. In particular, the FO–Elman model demonstrates improved tracking accuracy during rapid load variations, as illustrated in the highlighted interval of Fig. 21. These results indicate that fractional-order optimization enables recurrent networks with moderate complexity to effectively handle long-horizon and highly dynamic time-series prediction tasks.

5. Conclusion

This paper proposes a fractional-order gradient descent Elman neural network learning algorithm (FO-Elman) based on the Grünwald–Letnikov definition. By introducing a fractional-order gradient computation mechanism with memory characteristics, the proposed algorithm effectively alleviates the tendency of conventional integer-order gradient descent methods to be trapped in local optima and their strong sensitivity to the learning rate when modeling complex dynamic systems. In the system identification experiments, compared with conventional Elman networks optimized by SGDM, Adam, and RMSProp, FO-Elman consistently achieves lower steady-state errors across multiple test scenarios, with the MSE reduced by approximately 20%–40% on average. These performance gains are obtained while maintaining the same storage complexity, at the cost of a moderate and acceptable increase in training time. In the time series prediction tasks, FO-Elman attains prediction performance on the same order of magnitude as that of recurrent neural network models such as LSTM and GRU, while exhibiting clear advantages in terms of model size and training time.

These results indicate that FO-Elman is suitable for applications with limited model size or computational resources. Nevertheless, it should be noted that the incorporation of fractional-order gradient computations increases the computational burden during training, which may lead to additional training costs for large-scale datasets or long time series. Moreover, when extending the proposed fractional-order optimization mechanism to deeper or more complex network architectures, issues such as increased gradient accumulation overhead and greater difficulty in parameter tuning may arise, which can limit the scalability of the proposed method.

CRedit authorship contribution statement

He Li: Conceptualization, Funding acquisition, Methodology, Writing – original draft; **Shanze Wang:** Methodology, Software, Writing – original draft; **Yanguan Chen:** Conceptualization, Methodology; **Yanmei Liu:** Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

The author is an Editorial Board Member/Editor-in-Chief/Associate Editor/Guest Editor for this journal and was not involved in the editorial review or the decision to publish this article.

References

- Amalou, I., Mouhni, N., & Abdali, A. (2022). Multivariate time series prediction by RNN architectures for energy consumption forecasting. *Energy Reports*, 8, 1084–1091. <https://doi.org/10.1016/j.egy.2022.07.139>
- Atabay, F. V., Pagkalinawan, R. M., Pajarillo, S. D., Villanueva, A. R., & Taylar, J. V. (2022). Multivariate time series forecasting using ARIMAX, SARIMAX, and RNN-based deep learning models on electricity consumption. In *2022 3rd International informatics and software engineering conference (IISEC)* (pp. 1–6). <https://doi.org/10.1109/IISEC56263.2022.9998301>
- Bao Huy, T. H., Vo, D. N., Nguyen, K. P., Huynh, V. Q., Huynh, M. Q., & Truong, K. H. (2023). Short-term load forecasting in power system using CNN-LSTM neural network. In *2023 Asia meeting on environment and electrical engineering (EEE-AM)* (pp. 01–06). <https://doi.org/10.1109/EEE-AM58328.2023.10395221>
- Bingi, K., Bhukya, R., & Kasi, V. R. (2025). Forecasting of renewable energy using fractional-order neural networks. In *Fractional-order activation functions for neural networks : Case studies on forecasting wind turbines' generated power* (pp. 181–206). Cham: Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-88091-9_6
- Bonassi, F., Farina, M., Xie, J., & Scatoloni, R. (2022). On recurrent neural networks for learning-based control: Recent results and ideas for future developments. *Journal of Process Control* 114, 92–104. <https://doi.org/10.1016/j.jprocont.2022.04.011>
- Cao, J., Yuan, K., & Li, H.-x. (2006). Global asymptotical stability of recurrent neural networks with multiple discrete delays and distributed delays. *IEEE Transactions on Neural Networks*, 17(6), 1646–1651. <https://doi.org/10.1109/TNN.2006.881488>

- Chen, G., Liang, Y., Li, S., & Xu, Z. (2024). A novel gradient descent optimizer based on fractional order scheduler and its application in deep neural networks. *Applied Mathematical Modelling* 128, 26–57. <https://doi.org/10.1016/j.apm.2023.12.018>
- Chen, L., Guo, W., Gu, P., Lopes, A. M., Chu, Z., & Chen, Y. (2023). Stability and stabilization of fractional-order uncertain nonlinear systems with multiorder. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 70(2), 576–580. <https://doi.org/10.1109/TCSII.2022.3178280>
- Chen, P., Luo, Y., Zheng, W., Gao, Z., & Chen, Y. (2021). Fractional order active disturbance rejection control with the idea of cascaded fractional order integrator equivalence. *ISA Transactions*, 114, 359–369. <https://doi.org/10.1016/j.isatra.2020.12.030>
- Cui, C., He, M., Di, F., Lu, Y., Dai, Y., & Lv, F. (2020). Research on power load forecasting method based on LSTM model. In *2020 IEEE 5th information technology and mechatronics engineering conference (ITOE)* (pp. 1657–1660). <https://doi.org/10.1109/ITOE49072.2020.9141684>
- Dey, R., & Salem, F. M. (2017). Gate-variants of gated recurrent unit (GRU) neural networks. In *2017 IEEE 60th international midwest symposium on circuits and systems (MWS-CAS)* (pp. 1597–1600). <https://doi.org/10.1109/MWSCAS.2017.8053243>
- Dhaked, D. K. (2024). A review on various voltage boosting topology in DC-DC converter. *Recent Advances in Electrical & Electronic Engineering*, 17(6), 554–572. <https://doi.org/10.2174/2352096516666230901140600>
- Dhaked, D. K., Dadhich, S., & Birla, D. (2023a). Power output forecasting of solar photovoltaic plant using LSTM. *Green Energy and Intelligent Transportation*, 2(5), 100113. <https://doi.org/10.1016/j.geits.2023.100113>
- Dhaked, D. K., Kumar, P., & Ganguly, S. (2024). Development of data driven model for proton exchange membrane fuel cell using machine learning approaches. In *2024 IEEE 3rd international conference on control, instrumentation, energy & communication (CIEC)* (pp. 67–72). <https://doi.org/10.1109/CIEC59440.2024.10468283>
- Dhaked, D. K., Raman, L. K., Birla, D., & Dwivedi, M. (2023b). Identification and classification of faults using fuzzy logic controller in transmission line. *Journal of Interdisciplinary Mathematics*, 26(3), 417–430. <https://doi.org/10.47974/JIM-1672>
- Fei, J., & Lu, C. (2018). Adaptive sliding mode control of dynamic systems using double loop recurrent neural network structure. *IEEE Transactions on Neural Networks and Learning Systems*, 29(4), 1275–1286. <https://doi.org/10.1109/TNNLS.2017.2672998>
- Feng, L., Tung, F., Hajimirsadeghi, H., Ahmed, M. O., Bengio, Y., & Mori, G. (2024). Attention as an RNN. *arXiv:2405.13956*.
- Folgheraiter, M., Yskak, A., & Yessirkepov, S. (2023). One-shot bipedal robot dynamics identification with a reservoir-based RNN. *IEEE Access*, 11, 50180–50194. <https://doi.org/10.1109/ACCESS.2023.3277977>
- Fu, S., Ma, X., Zhan, Y., You, F., Peng, Q., Liu, T., Bailey, J., & Mandic, D. (2024). Finding core labels for maximizing generalization of graph neural networks. *Neural Networks*, 180, 106635. <https://doi.org/10.1016/j.neunet.2024.106635>
- Gawlikowski, J., Tassi, C. R. N., Ali, M., Lee, J., Humt, M., Feng, J., Kruspe, A., Triebel, R., Jung, P., Roscher, R., Shahzad, M., Yang, W., Bamler, R., & Zhu, X. X. (2023). A survey of uncertainty in deep neural networks. *Artificial Intelligence Review* 56(1), 1513–1589. <https://doi.org/10.1007/s10462-023-10562-9>
- Gelly, G., & Gauvain, J.-L. (2018). Optimization of RNN-based speech activity detection. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 26(3), 646–656. <https://doi.org/10.1109/TASLP.2017.2769220>
- Greff, K., Srivastava, R. K., Koutník, J., Steunebrink, B. R., & Schmidhuber, J. (2017). LSTM: A search space odyssey. *IEEE Transactions on Neural Networks and Learning Systems* 28(10), 2222–2232. <https://doi.org/10.1109/TNNLS.2016.2582924>
- Hansun, S., & Young, J. C. (2021). Predicting LQ45 financial sector indices using RNN-LSTM. *Journal of Big Data*, 8(1), 104. <https://doi.org/10.1186/s40537-021-00495-x>
- Huang, J., & Qin, R. (2024). Elman neural network considering dynamic time delay estimation for short-term forecasting of offshore wind power. *Applied Energy* 358, 122671. <https://doi.org/10.1016/j.apenergy.2024.122671>
- Kang, Q., Li, X., Zhao, K., Cui, W., Zhao, Y., Deng, W., & Tay, W. P. (2025). Efficient training of neural fractional-order differential equation via adjoint backpropagation. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 17750–17759). (vol. 39).
- Khan, S., Ahmad, J., Naseem, I., & Moinuddin, M. (2018). A novel fractional gradient-based learning algorithm for recurrent neural networks. *Circuits, Systems, and Signal Processing*, 37(2), 593–612. <https://doi.org/10.1007/s00034-017-0572-z>
- King, M., Woo, S. I., & Yune, C.-Y. (2024). Utilizing a CNN-RNN machine learning approach for forecasting time-series outlet fluid temperature monitoring by long-term operation of BHEs system. *Geothermics*, 122, 103082. <https://doi.org/10.1016/j.geothermics.2024.103082>
- Kong, W., Dong, Z. Y., Jia, Y., Hill, D. J., Xu, Y., & Zhang, Y. (2019). Short-term residential load forecasting based on LSTM recurrent neural network. *IEEE Transactions on Smart Grid* 10(1), 841–851. <https://doi.org/10.1109/TSG.2017.2753802>
- Kosmatopoulos, E. B., Ioannou, P. A., & Christodoulou, M. A. (1992). Identification of nonlinear systems using new dynamic neural network structures. In *1992 Proceedings of the 31st IEEE conference on decision and control* (pp. 20–25 vol.1). <https://doi.org/10.1109/CDC.1992.371800>
- Kumar, R. (2024). Recurrent context layered radial basis function neural network for the identification of nonlinear dynamical systems. *Neurocomputing*, 580, 127524. <https://doi.org/10.1016/j.neucom.2024.127524>
- Kuschewski, J. G., Hui, S., & Zak, S. H. (1993). Application of feedforward neural networks to dynamical system identification and control. *IEEE Transactions on Control Systems Technology*, 1(1), 37–49. <https://doi.org/10.1109/87.221350>
- Lakshmi Narayanan, V., Kumar Dhaked, D., & Sitharthan, R. (2024). Improved machine learning-based pitch controller for rated power generation in large-scale wind turbine. *Renewable Energy Focus*, 50, 100603. <https://doi.org/10.1016/j.ref.2024.100603>
- Lanzetti, N., Lian, Y. Z., Cortinovis, A., Dominguez, L., Mercangöz, M., & Jones, C. (2019). Recurrent neural network based MPC for process industries. In *2019 18th European control conference (ECC)* (pp. 1005–1010). <https://doi.org/10.23919/ECC.2019.8795809>

- Liou, C.-Y., Huang, J.-C., & Yang, W.-C. (2008). Modeling word perception using the Elman network. *Neurocomputing*, 71(16), 3150–3157. <https://doi.org/10.1016/j.neucom.2008.04.030>
- Liu, H., Tian, H.-q., Liang, X.-f., & Li, Y.-f. (2015). Wind speed forecasting approach using secondary decomposition algorithm and elman neural networks. *Applied Energy*, 157, 183–194. <https://doi.org/10.1016/j.apenergy.2015.08.014>
- Liu, P., Nie, X., Liang, J., & Cao, J. (2018). Multiple mittag-leffler stability of fractional-order competitive neural networks with gaussian activation functions. *Neural Networks*, 108, 452–465. <https://doi.org/10.1016/j.neunet.2018.09.005>
- Liu, S., Zhang, L., & Xia, H. (2022). Research on power load prediction based on Elman network with improved particle swarm optimization. *Journal of Physics: Conference Series*, 2338(1), 012076. <https://doi.org/10.1088/1742-6596/2338/1/012076>
- Liu, W., Zheng, J., Shi, G., Yuan, Q., & Lu, Y. (2025). Research on parameter identification and fault prediction method of hydraulic system in intelligent sensing agriculture. *Measurement: Sensors*, 38, 101813. <https://doi.org/10.1016/j.measen.2025.101813>
- Liu, Z., Cao, J., You, J., Chen, S., Lu, Y., & Zhou, P. (2021). A lithological sequence classification method with well log via SVM-assisted bi-directional GRU-CRF neural network. *Journal of Petroleum Science and Engineering*, 205, 108913. <https://doi.org/10.1016/j.petrol.2021.108913>
- Lopes Ferrari, A., Carolina Sérgio Gomes, M., Caroline Raimundini Aranha, A., Marques Paschoal, S., de Souza Matias, G., Mario de Matos Jorge, L., & Oliveira Defendi, R. (2024). Mathematical modeling by fractional calculus applied to separation processes. *Separation and Purification Technology* 337, 126310. <https://doi.org/10.1016/j.seppur.2024.126310>
- Lyu, C., Chen, B., Ren, Y., & Ji, D. (2017). Long short-term memory RNN for biomedical named entity recognition. *BMC Bioinformatics*, 18(1), 462. <https://doi.org/10.1186/s12859-017-1868-5>
- López-Caraballo, C. H., Salfate, I., Lazzús, J. A., Rojas, P., Rivera, M., & Palma-Chilla, L. (2016). Mackey-glass noisy chaotic time series prediction by a swarm-optimized neural network. *Journal of Physics: Conference Series*, 720(1), 012002. <https://doi.org/10.1088/1742-6596/720/1/012002>
- Naderi, M. A., & Mahdavi-Nasab, H. (2010). Analysis and classification of EEG signals using spectral analysis and recurrent neural networks. In *2010 17th Iranian conference of biomedical engineering (ICBME)* (pp. 1–4). <https://doi.org/10.1109/ICBME.2010.5704931>
- Pang, G., Lu, L., & Karniadakis, G. E. (2019). fPINNs: Fractional physics-informed neural networks. *SIAM Journal on Scientific Computing*, 41(4), A2603–A2626. <https://doi.org/10.1137/18M1229845>
- Ramakrishnan, N., & Soni, T. (2018). Network traffic prediction using recurrent neural networks. In *2018 17th IEEE international conference on machine learning and applications (ICMLA)* (pp. 187–193). <https://doi.org/10.1109/ICMLA.2018.00035>
- Saon, G., Tüske, Z., Bolanos, D., & Kingsbury, B. (2021). Advancing RNN transducer technology for speech recognition. In *ICASSP 2021 - 2021 IEEE international conference on acoustics, speech and signal processing (ICASSP)* (pp. 5654–5658). <https://doi.org/10.1109/ICASSP39728.2021.9414716>
- Sharma, P., Dhaked, D. K., & Sharma, A. K. (2021). Mathematical modeling and simulation of DC-DC converters using state-space approach. In D. Goyal, P. Chaturvedi, A. K. Nagar, & S. D. Purohit (Eds.), *Proceedings of second international conference on smart energy and communication* (pp. 11–29). Singapore: Springer Singapore.
- Sivalingam, S. M., & Govindaraj, V. (2025). Neural fractional order differential equations. *Expert Systems with Applications* 267, 126041. <https://doi.org/10.1016/j.eswa.2024.126041>
- Soydaner, D. (2022). Attention mechanism in neural networks: Where it comes and where it goes. *Neural Computing and Applications*, 34(16), 13371–13385. <https://doi.org/10.1007/s00521-022-07366-3>
- Sun, Y., Zhou, Q., Sun, L., Sun, L., Kang, J., & Li, H. (2024). CNN-LSTM-AM: A power prediction model for offshore wind turbines. *Ocean Engineering*, 301, 117598. <https://doi.org/10.1016/j.oceaneng.2024.117598>
- The MathWorks, I. (2024). Matlab version: 9.13.0 (r2024b). <https://www.mathworks.com>.
- Tian, Y., & Pan, L. (2015). Predicting short-term traffic flow by long short-term memory recurrent neural network. In *2015 IEEE International conference on smart city/social-com/sustaincom (smartcity)* (pp. 153–158). <https://doi.org/10.1109/SmartCity.2015.63>
- Vellappandi, M., & Lee, S. (2025). Neural fractional differential networks for modeling complex dynamical systems. *Nonlinear Dynamics*, 113(10), 12117–12130. <https://doi.org/10.1007/s11071-024-10795-5>
- Wang, J., Li, X., Li, J., Sun, Q., & Wang, H. (2022). NGCU: A new RNN model for time-series data prediction. *Big Data Research*, 27, 100296. <https://doi.org/10.1016/j.bdr.2021.100296>
- Wang, J., Wen, Y., Gou, Y., Ye, Z., & Chen, H. (2017). Fractional-order gradient descent learning of BP neural networks with caputo derivative. *Neural Networks*, 89, 19–30. <https://doi.org/10.1016/j.neunet.2017.02.007>
- Wang, J.-S., & Chen, Y.-P. (2006). A fully automated recurrent neural network for unknown dynamic system identification and control. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 53(6), 1363–1372. <https://doi.org/10.1109/TCSI.2006.875186>
- Yao, X., & Wang, Z. (2020). Fractional order echo state network for time series prediction. *Neural Processing Letters*, 52(1), 603–614. <https://doi.org/10.1007/s11063-020-10267-y>
- Yuan, M., Li, Y., & Zheng, M. (2026). Novel intermittent pinning control beyond the semi-group framework for fractional-order delayed inertial memristive neural networks. *Chaos, Solitons & Fractals*, 202, 117568.
- Zarzycki, K., & Ławryńczuk, M. (2021). LSTM and GRU neural networks as models of dynamical processes used in predictive control: A comparison of models developed for two chemical reactors. *Sensors*, 21(16). <https://doi.org/10.3390/s21165625>
- Zha, W., Liu, Y., Wan, Y., Luo, R., Li, D., Yang, S., & Xu, Y. (2022). Forecasting monthly gas field production based on the CNN-LSTM model. *Energy*, 260, 124889. <https://doi.org/10.1016/j.energy.2022.124889>
- Zhang, H., Pu, Y.-F., Xie, X., Zhang, B., Wang, J., & Huang, T. (2021a). A global neural network learning machine: Coupled integer and fractional calculus operator with an adaptive learning scheme. *Neural Networks*, 143, 386–399. <https://doi.org/10.1016/j.neunet.2021.06.021>
- Zhang, J., Liu, T., & Tao, D. (2024a). Going deeper, generalizing better: An information-theoretic view for deep learning. *IEEE Transactions on Neural Networks and Learning Systems*, 35(11), 16683–16695. <https://doi.org/10.1109/TNNLS.2023.3297113>
- Zhang, S., Zhao, Z., Wu, J., Jin, Y., Jeng, D.-S., Li, S., Li, G., & Ding, D. (2024b). Solving the temporal lags in local significant wave height prediction with a new VMD-LSTM model. *Ocean Engineering*, 313, 119385. <https://doi.org/10.1016/j.oceaneng.2024.119385>
- Zhang, Y., Zhao, J., Wang, L., Wu, H., Zhou, R., & Yu, J. (2021b). An improved OIF elman neural network based on CSO algorithm and its applications. *Computer Communications*, 171, 148–156. <https://doi.org/10.1016/j.comcom.2021.01.035>
- Zhao, B., Li, X., Lu, X., & Wang, Z. (2018). A CNN-RNN architecture for multi-label weather recognition. *Neurocomputing*, 322, 47–57. <https://doi.org/10.1016/j.neucom.2018.09.048>
- Zhao, J., Zeng, D., Liang, S., Kang, H., & Liu, Q. (2021). Prediction model for stock price trend based on recurrent neural network. *Journal of Ambient Intelligence and Humanized Computing*, 12(1), 745–753. <https://doi.org/10.1007/s12652-020-02057-0>
- Zhao, Y., Jin, X., & Hu, X. (2017). Recurrent convolutional neural network for speech processing. In *2017 IEEE International conference on acoustics, speech and signal processing (ICASSP)* (pp. 5300–5304). <https://doi.org/10.1109/ICASSP.2017.7953168>
- Zuñiga Aguilar, C. J., Gómez-Aguilar, J. F., Alvarado-Martínez, V. M., & Romero-Ugalde, H. M. (2020). Fractional order neural networks for system identification. *Chaos, Solitons & Fractals*, 130, 109444. <https://doi.org/10.1016/j.chaos.2019.109444>